Java Multithreading & Concurrency — Senior Interview Master Notes (2026)

15 YOE | Print-Ready | Descending Interview Frequency

PRIORITY MAP

#	Topic	Frequency	Java Version
1	Virtual Threads & Project Loom	95% 🔥	Java 21 GA
2	CompletableFuture — async chaining	92% 🔥	Java 8+
3	Thread Safety (sync/volatile/atomic)	90% 🔥	All
4	Deadlock / Race / Starvation	88% 🔥	All
5	ExecutorService & ThreadPoolExecutor	85% 🔥	Java 5+
6	Java Memory Model (JMM)	80% 🔥	All
7	wait/notify / Producer-Consumer	78% ✅	All
8	Structured Concurrency	70% ✅	Java 25 GA
9	CountDownLatch / CyclicBarrier / Semaphore	68% ✅	Java 5+
10	Fork/Join & parallel streams	60% ✅	Java 7+
11	ThreadLocal — tracing & memory leaks	58% ✅	All
12	Thread fundamentals (start/run/join/daemon)	55% 👍	All
13	Spring @Async + MDC propagation	55% 👍	Spring 4+
14	LongAdder vs AtomicInteger	45% 👍	Java 8+
15	Non-blocking vs Async (WebClient/Reactor)	50% 👍	Spring 5+
16	Concurrent Collections (CHM/CopyOnWrite/BQ)	65% ✅	Java 5+
17	ReentrantLock + Condition	62% ✅	Java 5+
18	Synchronized same-object lock gotcha	70% ✅	All
19	ScopedValue (Java 25 GA)	60% ✅	Java 25

1. VIRTUAL THREADS & PROJECT LOOM (95% — MUST KNOW)

The Problem with Platform Threads

```
10,000 clients + pool of 200 threads = 9,800 requests queued
Each OS thread ≈ 1 MB stack → 200 threads ≈ 200 MB just for pool
```

```
                    PLATFORM THREADS (old way)
                    ──────────────────────────

  HTTP Request ──►  Thread-1   ──► calls DB  ──► WAITING 200ms
  HTTP Request ──►  Thread-2   ──► calls API ──► WAITING 500ms
  HTTP Request ──►  Thread-3   ──► calls DB  ──► WAITING 200ms
  ...
  HTTP Request ──►  Thread-200 ──► calls DB  ──► WAITING 200ms
  HTTP Request ──►  [QUEUED]   ──► waiting for a free thread!
  HTTP Request ──►  [QUEUED]
  HTTP Request ──►  [QUEUED]   ← 9800 requests stuck!

  All 200 threads are ALIVE but doing nothing — just waiting on I/O.
  OS thread is blocked. 200MB RAM wasted on idle stacks.
```

How Virtual Threads Fix This

```
                    VIRTUAL THREADS (new way)
                    ─────────────────────────

  JVM has ~8 "carrier threads" (= CPU cores)

  Virtual Thread-1    ──► calls DB  ──► [PARKED by JVM]
                                              │
                                carrier freed ──► picks up VT-5001
  Virtual Thread-5001 ──► runs      ──► calls API ──► [PARKED]
                                              │
                                carrier freed ──► picks up VT-2
  ...

  1,000,000 virtual threads? Fine.
  Each parks when it blocks. Carrier thread never idles.
  Memory per VT: ~few KB (vs 1 MB for OS thread)
```

Internal Lifecycle on Blocking I/O

```
  VT calls Thread.sleep() or socket.read()
       │
       ▼
  JVM intercepts (not OS!)
       │
       ▼
```

```
  VT state saved to HEAP (cheap — just an object)
       │
       ▼
  Carrier thread UNMOUNTED from VT
       │
       ▼
  Carrier thread picks up next RUNNABLE virtual thread
       │
       ▼
  When I/O completes → VT marked RUNNABLE → mounted back onto a carrier
```

Production Impact

```
Scenario: REST service, each request does 3 DB calls × 50ms each

  Platform threads (pool=200):
    Throughput = 200 / 150ms = ~1,333 req/sec
    At 2,000 req/sec → queue backs up → latency spike → timeouts

  Virtual threads:
    Throughput limited by DB connection pool, not thread count
    10,000 req/sec? Carrier threads just keep parking and resuming VTs
    Memory: same 8 carrier threads + tiny VT heap objects
```

Three Ways to Create Virtual Threads

```
// Way 1: Thread.ofVirtual (simplest)
Thread vt = Thread.ofVirtual().start(() -> System.out.println("virtual"));

// Way 2: VirtualThreadPerTaskExecutor (recommended for pools)
try (ExecutorService executor =
Executors.newVirtualThreadPerTaskExecutor()) {
    for (int i = 0; i < 1_000_000; i++) {
        executor.submit(() -> {
            Thread.sleep(2000);   // blocks cheaply — JVM parks, not OS
            callExternalAPI();
        });
    }
}

// Way 3: Thread factory (Spring Boot 3.2+ integration)
@Bean
public TomcatProtocolHandlerCustomizer<?> virtualThreads() {
    return handler ->
handler.setExecutor(Executors.newVirtualThreadPerTaskExecutor());
}
```

Key Facts

Aspect	Platform Thread	Virtual Thread
Memory	~1 MB stack	~few KB
Max count	~10,000	Millions
Blocking	Wastes OS thread	JVM parks it
Creation cost	High	Near-zero
When to use	CPU-bound work	I/O-bound work

Pinning — The Critical Gotcha

Virtual thread gets **pinned** (can't unmount from carrier thread) when:

1. Inside `synchronized` block doing blocking I/O
2. Calling native methods

```
// ❌ PINNING — blocks carrier thread
synchronized(lock) {
    Thread.sleep(1000);       // pin!
    Files.readAllBytes(path); // pin!
}

// ✅ NO PINNING — use ReentrantLock instead
ReentrantLock lock = new ReentrantLock();
lock.lock();
try {
    Thread.sleep(1000);       // safe — virtual thread parks
} finally {
    lock.unlock();
}
```

Diagnose pinning: `-Djdk.tracePinnedThreads=full`

Interview One-Liner

> "Virtual threads are JVM-managed, not OS-managed. When they block on I/O, JVM unmounts them from the carrier thread — freeing that carrier to run other virtual threads. This gives you blocking code style with reactive-level scalability. Watch for pinning inside `synchronized` blocks."

---

2. COMPLETABLEFUTURE (92% — MUST KNOW)

The Problem

`Future.get()` is a blocking call — the thread that calls it sits idle until the result is ready. With a pool of 200 threads each making a 500ms external API call via `Future.get()`, all 200 threads are simultaneously blocked; the pool exhausts and new requests queue up or are rejected outright.

What Happens Without It

```
Scenario: 200-thread pool, each task calls an external API (avg 500 ms
latency)

  Thread-1   → future.get() → BLOCKED 500 ms ──┐
  Thread-2   → future.get() → BLOCKED 500 ms   │ all 200 threads
  Thread-3   → future.get() → BLOCKED 500 ms   │ doing ZERO CPU work
  ...                                          │
  Thread-200 → future.get() → BLOCKED 500 ms ──┘

  Request-201 → RejectedExecutionException  ← no free threads

  200 threads × 1 MB stack = 200 MB RAM consumed while idle.
  Throughput ceiling: 200 / 0.5 s = 400 req/s — hard wall.
  Any latency spike above 500 ms → pool exhaustion → cascading failure.
```

Why CompletableFuture

Attach a callback that fires when the result arrives instead of holding a thread hostage waiting for it.

Core Mental Model

```
Future.get()          → BLOCKS the calling thread
CompletableFuture     → attaches callbacks, NEVER blocks (unless you call
.get()/.join())
```

Essential API Cheatsheet

```
// Start async computation
CompletableFuture<String> cf = CompletableFuture.supplyAsync(() ->
fetchData());

// Transform result (thenApply = map)
cf.thenApply(data -> data.toUpperCase());

// Consume result (thenAccept = forEach)
cf.thenAccept(System.out::println);

// Chain another async task (thenCompose = flatMap)
cf.thenCompose(data -> CompletableFuture.supplyAsync(() -> process(data)));

// Combine two independent futures
CompletableFuture<String>  priceFuture    =
CompletableFuture.supplyAsync(() -> getPrice());
CompletableFuture<Integer> inventoryFuture =
CompletableFuture.supplyAsync(() -> getStock());
priceFuture.thenCombine(inventoryFuture, (price, stock) -> price + " / " +
```

```
stock);

// Wait for ALL
CompletableFuture.allOf(cf1, cf2, cf3).thenRun(() ->
System.out.println("all done"));

// First to complete wins
CompletableFuture.anyOf(cf1, cf2, cf3).thenAccept(System.out::println);

// Error handling
cf.exceptionally(ex -> "default")
  .whenComplete((result, ex) -> log(result, ex));
```

Real-World: Order Processing (parallel fan-out)

```
public CompletableFuture<OrderConfirmation> processOrder(Order order) {
    CompletableFuture<Boolean> inventory = CompletableFuture.supplyAsync(()
-> checkInventory(order));
    CompletableFuture<Boolean> payment   = CompletableFuture.supplyAsync(()
-> validatePayment(order));
    CompletableFuture<Boolean> fraud     = CompletableFuture.supplyAsync(()
-> detectFraud(order));

    return CompletableFuture.allOf(inventory, payment, fraud)
        .thenApply(v -> {
            if (inventory.join() && payment.join() && !fraud.join()) {
                return new OrderConfirmation("APPROVED");
            }
            return new OrderConfirmation("REJECTED");
        })
        .exceptionally(ex -> new OrderConfirmation("ERROR"));
}
```

Execution Flow — thenApply Chain

```
  MAIN THREAD                    FORKJOIN WORKER                    FORKJOIN
WORKER
  ___________                    _______________
_______________
  supplyAsync(supplier)
       |
       ▼
  CF<String> cf ◄── returned immediately
  (CF not yet complete)
                                 scheduler picks up task
                                        |
                                        ▼
                                 supplier runs
                                 (HTTP call, DB read...)
```

```
                                    |
                                    ▼
                                cf.complete("result")
                                    |
                                thenApply callback fires
                                ON THE SAME WORKER
                                (worker that completed cf)
                                    |
                                    ▼
                                transform("result") → "RESULT"
                                    |
                                thenAccept callback fires
                                STILL ON SAME WORKER
                                    |
                                    ▼
                                println("RESULT")

  ──────────────────────────────────────────────────────────
  Stage ownership rules:
  ──────────────────────────────────────────────────────────
  thenApply / thenAccept     → runs on thread that COMPLETED prev stage
  thenApplyAsync             → runs on new ForkJoin worker (or custom pool)
  CF already done at attach  → callback runs on CALLER thread

  ┌──────────────────────────────────────────────────────────┐
  │  supplyAsync    ──► Worker-3 executes supplier           │
  │       |                                                  │
  │       ▼                                                  │
  │  thenApply      ──► Worker-3 runs transform (same)       │
  │       |                                                  │
  │       ▼                                                  │
  │  thenApplyAsync ──► Worker-7 runs next stage (new)       │
  │       |                                                  │
  │       ▼                                                  │
  │  thenAccept     ──► Worker-7 runs consumer (same as prev)│
  └──────────────────────────────────────────────────────────┘
```

Common Mistakes

```
// ❌ WRONG — blocks the thread, wastes async benefits
String result = cf.get();   // blocking

// ❌ WRONG — exception swallowed silently
cf.thenApply(data -> riskyOp(data));  // no error handler

// ✅ RIGHT — always handle exceptions
cf.thenApply(data -> riskyOp(data))
  .exceptionally(ex -> fallback());
```

Interview One-Liner

> "CompletableFuture is a monad — it lets you chain async operations without blocking. thenApply transforms, thenCompose flatMaps, allOf fans out. Always add exceptionally() or you'll lose exceptions silently."

---

3. THREAD SAFETY (90% — MUST KNOW)

The Problem

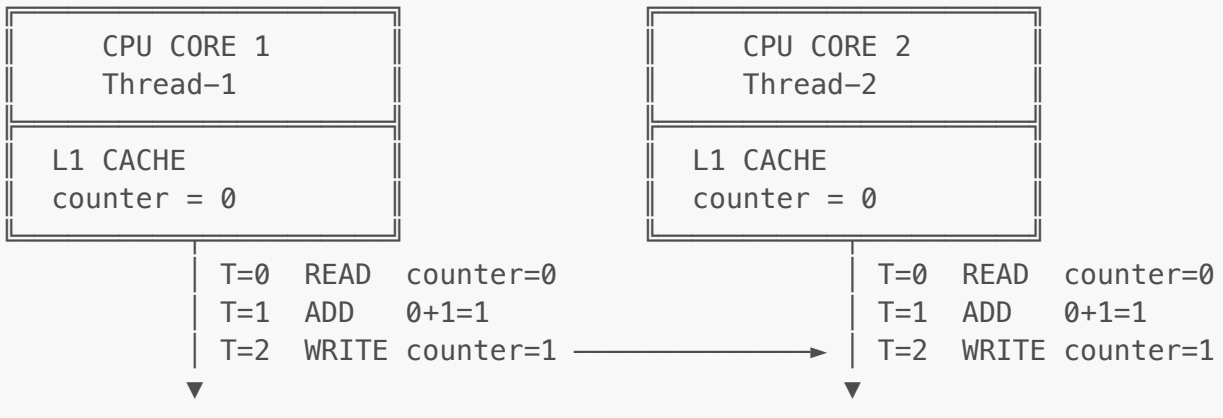
Any field that is both shared across threads and mutated by at least one of them is a race condition waiting to happen. Without explicit protection, the JVM and CPU are free to reorder, cache, and interleave reads and writes in ways that produce corrupt state — silently, with no exception.

What Happens Without It

```
10 threads each call counter++ 1,000 times.
Expected final value: 10,000.

  Run 1 → 9,743
  Run 2 → 8,991
  Run 3 → 9,512   ← different each time, always wrong

Why: counter++ compiles to 3 bytecode instructions (READ / MODIFY / WRITE).
Thread can be preempted between READ and WRITE.
The writing thread overwrites another thread's increment — update is LOST.
No exception. No log line. The value is just wrong.
```

The Race Condition Root Cause

```
counter++  is NOT atomic — it's: READ → MODIFY → WRITE (3 instructions)

T1: READ(0) → ...preempted...
T2: READ(0) → MODIFY(1) → WRITE(1)
T1: ...resumed... MODIFY(1) → WRITE(1)
Result: 1 instead of 2!
```



```
┌──────────────────────────────────────────────────────┐
│                    MAIN MEMORY                       │
│   counter = 1  ◄── Core 2 write arrives last, wins   │
│   Core 1 wrote 1. Core 2 wrote 1. Result: 1. Expected: 2.  │
│   ONE UPDATE PERMANENTLY LOST.                       │
└──────────────────────────────────────────────────────┘
```

Five Strategies (choose by use case)

Strategy 1 — synchronized (simplest, coarse)

```
private int counter = 0;

public synchronized void increment() { counter++; }  // method-level
public void increment2() {
    synchronized(this) { counter++; }                 // block-level (fine-grained)
}
```

Strategy 2 — AtomicInteger (best for counters)

```
private final AtomicInteger counter = new AtomicInteger(0);

counter.incrementAndGet();          // atomic read-modify-write
counter.compareAndSet(5, 10);       // CAS — only sets if current == 5
counter.getAndUpdate(x -> x * 2);   // any function atomically
```

Strategy 3 — volatile (flags/signals only)

```
private volatile boolean running = true;  // visibility guarantee

// ❌ WRONG — volatile ≠ atomicity
volatile int count = 0;
count++;  // still a race condition!

// ✅ Use volatile only for single-write, multi-read flags
```

Strategy 4 — ConcurrentHashMap

```
Map<String, Integer> map = new ConcurrentHashMap<>();
map.putIfAbsent("key", 1);           // atomic check-then-act
map.compute("key", (k, v) -> v + 1); // atomic update
```

Strategy 5 — Immutability (best design)

```
public final class Money {
    private final long amount;
    private final String currency;
    // constructor only, no setters — inherently thread-safe
}
```

volatile vs AtomicInteger Decision Table

Need	Use
Shutdown flag, feature toggle	`volatile boolean`
Request counter, ID generator	`AtomicInteger`
Multi-step compound operations	`synchronized` or `Lock`
High-contention counter	`LongAdder`

Interview One-Liner

> "Thread safety = protect shared mutable state. volatile gives visibility, not atomicity. AtomicInteger gives both for single variables via CAS. For compound actions, use synchronized or ReentrantLock. Best solution: eliminate shared mutable state via immutability."

---

4. DEADLOCK / RACE / STARVATION (88% — MUST KNOW)

The Problem

Two or more threads permanently block each other — each holds a lock the other needs. No exception is raised, no timeout fires, no log entry appears. The JVM parks those threads forever while the rest of the pool drains into the same trap.

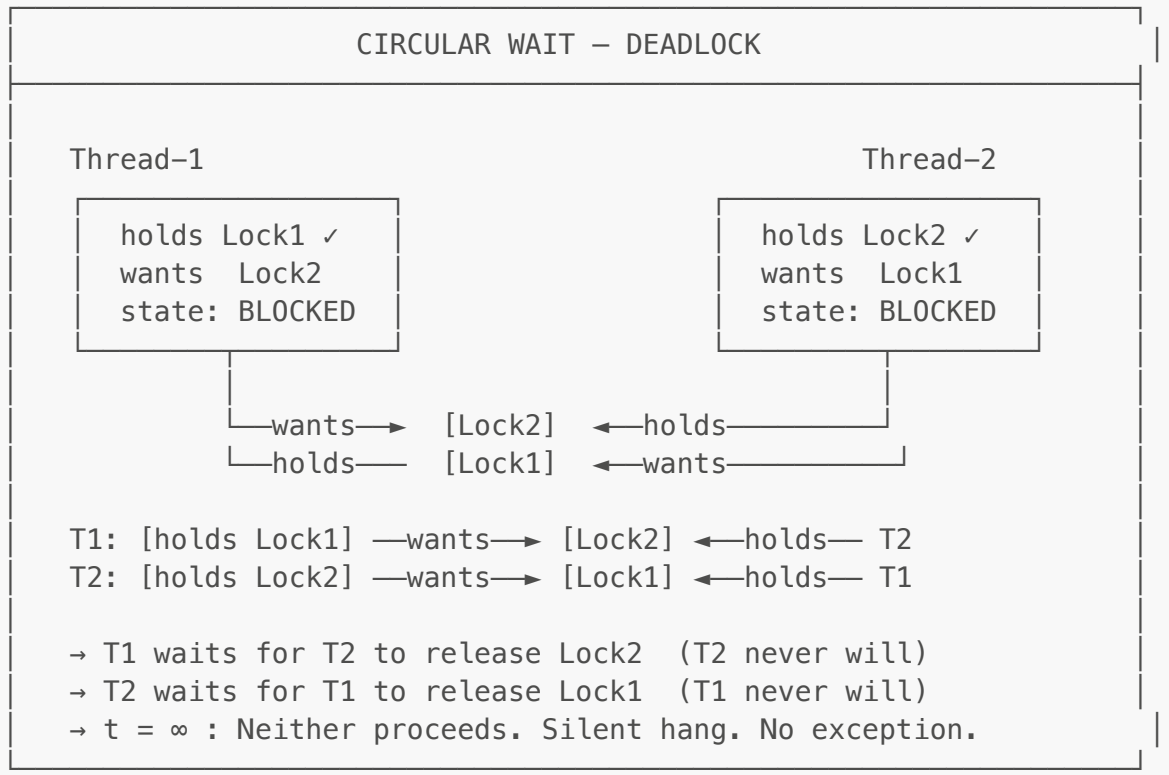
What Happens Without Prevention

A payment service acquires a row-lock on the `orders` table, then calls inventory over HTTP. Simultaneously, inventory acquires the same row-lock and calls payment back. Both threads park indefinitely. All 200 pool threads hit the same pattern within seconds. CPU drops to 0%, heap looks fine, health-check returns 200 — yet no request completes. No `DeadlockException` is thrown. Only a thread dump reveals the circular wait chain.

Deadlock — Four Necessary Conditions (ALL must be true)

1. **Mutual Exclusion** — resource held exclusively
2. **Hold & Wait** — holds one, waits for another
3. **No Preemption** — lock not forcibly taken
4. **Circular Wait** — T1→lock2, T2→lock1

```
// ❌ DEADLOCK
synchronized(lock1) {
    synchronized(lock2) { ... }  // T1: holds lock1, wants lock2
}
synchronized(lock2) {
    synchronized(lock1) { ... }  // T2: holds lock2, wants lock1
}

// ✅ FIX 1: Always acquire locks in the SAME order
synchronized(lock1) {
    synchronized(lock2) { ... }  // Both T1 and T2 take lock1 first
}

// ✅ FIX 2: tryLock with timeout
if (lock1.tryLock(1, TimeUnit.SECONDS)) {
    try {
        if (lock2.tryLock(1, TimeUnit.SECONDS)) {
            try { /* safe */ } finally { lock2.unlock(); }
        }
    } finally { lock1.unlock(); }
}

// ✅ FIX 3: avoid nested locks entirely — use higher-level abstractions
```



Execution Flow — How Deadlock Forms

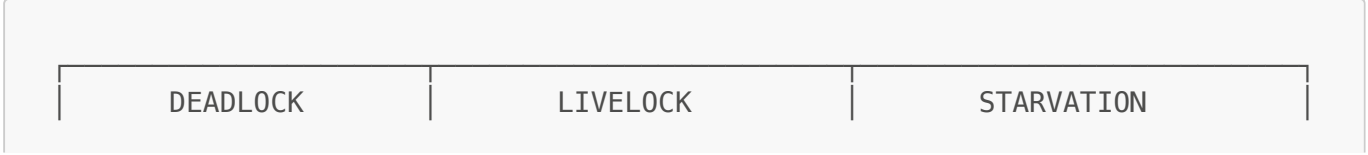
```
t=0ms ── T1 acquires Lock1
──────────────────────────────────────────────────────────────────
T1: [Lock1 ✓]                    T2: [no locks, running]

t=1ms ── T2 acquires Lock2
──────────────────────────────────────────────────────────────────
T1: [Lock1 ✓]                    T2: [Lock2 ✓]

t=2ms ── T1 tries Lock2 → BLOCKED (T2 holds it)
──────────────────────────────────────────────────────────────────
T1: [Lock1 ✓] ──wants──► [Lock2] ▼ BLOCKED
T2:                       [Lock2 ✓]  still running

t=3ms ── T2 tries Lock1 → BLOCKED (T1 holds it)
──────────────────────────────────────────────────────────────────
T1: [Lock1 ✓] ──wants──► [Lock2] ▼ BLOCKED
T2: [Lock2 ✓] ──wants──► [Lock1] ▼ BLOCKED

t=∞ ─── Neither thread can proceed.
──────────────────────────────────────────────────────────────────
CPU:          0%   (threads parked, no spinning)
Heap:         fine (no allocation happening)
Health check: 200 OK (process is alive)
Symptom:      requests queue up → pool exhausts → timeouts begin
Detection:    jstack <pid> → "waiting to lock" → circular chain
```

Detect Deadlock in Production

```
# Thread dump
kill -3 <pid>        # Linux/Mac
jstack <pid>         # JDK tool

# Look for: "waiting to lock" + circular dependencies in dump
```

Livelock vs Starvation

```
Deadlock:   T1 and T2 both BLOCKED forever
Livelock:   T1 and T2 keep RUNNING but making no progress (both yield to
each other)
Starvation: T3 always loses to high-priority T1, T2 — never gets CPU

Fix starvation: ReentrantLock(true)  // fair lock — FIFO ordering
```



```
┌──────────────────────┬──────────────────────┬──────────────────────────┐
│ T1 ──► [BLOCKED]     │ T1 ►yield──► T2      │ T1 (high pri) ──────► runs │
│ T2 ──► [BLOCKED]     │ T2 ►yield──► T1      │ T2 (high pri) ──────► runs │
│                      │ T1 ►yield──► T2 ...  │ T3 (low  pri) ──► waits  │
│ State:  BLOCKED      │ State:  RUNNING      │ State: RUNNABLE, never   │
│ CPU:    0%           │ CPU:    100% (wasted)│ scheduled                │
│ Progress: none       │ Progress: none       │ CPU:    T3 gets ~0%      │
│                      │                      │ Progress: T3 = none      │
├──────────────────────┼──────────────────────┼──────────────────────────┤
│ Fix: consistent      │ Fix: random backoff  │ Fix:                     │
│   lock ordering      │   or jitter before   │   ReentrantLock(true)    │
│   or tryLock timeout │   each retry         │   (FIFO fair ordering)   │
└──────────────────────┴──────────────────────┴──────────────────────────┘
```

Interview One-Liner

> "Deadlock needs all four conditions — break any one to prevent it. The safest strategy is consistent lock ordering. For detection, take thread dumps and look for circular wait chains. Use tryLock with timeout as a runtime safeguard."

---

5. EXECUTORSERVICE & THREAD POOLS (85% — MUST KNOW)

The Problem

Without a pool, a naive thread-per-request model creates one OS thread per incoming request. Each OS thread consumes ~1 MB of stack. 10,000 simultaneous requests = 10,000 threads ≈ 10 GB of stack space — JVM throws `OutOfMemoryError: unable to create native thread` before the load spike peaks. Thread creation itself costs ~50–100 µs, adding latency before any work begins.

What Happens Without It

```
Anti-pattern: Executors.newCachedThreadPool() under traffic spike
SynchronousQueue — zero buffer, spawns a NEW thread for every submitted
task

  Requests/sec │  Threads created │  Stack memory  │  Outcome
 ──────────────┼──────────────────┼────────────────┼──────────────────────
     100       │       100        │    ~100 MB     │  fine
   1,000       │     1,000        │    ~  1 GB     │  sluggish
   5,000       │     5,000        │    ~  5 GB     │  GC pressure, pauses
  10,000       │    10,000        │    ~ 10 GB     │  OutOfMemoryError ✗
```

Real incident: a marketing email blast triggered 8,000 concurrent HTTP callbacks into a `newCachedThreadPool`. JVM threw `OutOfMemoryError` in 4 seconds. Zero requests completed.

Why Thread Pools

- **Reuse** — OS threads cost ~50–100 µs + 1 MB to create; pools amortise that cost to near-zero

- **Control** — cap concurrency so DB connections, file handles, and downstream services are not overwhelmed
- **Backpressure** — a bounded queue signals upstream to slow down instead of silently dropping work
- **Observability** — queue depth, active-thread count, and rejection rate are measurable and alertable

Factory Methods vs Custom Configuration

```
// Factory shortcuts (avoid in production — unbounded queues)
Executors.newFixedThreadPool(10)        // fixed size, unbounded LinkedBlockingQueue
Executors.newCachedThreadPool()         // grows without bound — DANGEROUS
Executors.newSingleThreadExecutor()     // 1 thread, ordered execution
Executors.newScheduledThreadPool(5)     // cron-like scheduling

// ✅ Production — always use ThreadPoolExecutor directly
ThreadPoolExecutor executor = new ThreadPoolExecutor(
    10,                                 // corePoolSize
    50,                                 // maximumPoolSize
    60L, TimeUnit.SECONDS,              // keepAlive for surplus threads
    new ArrayBlockingQueue<>(200),      // BOUNDED queue — prevents OOM
    new ThreadPoolExecutor.CallerRunsPolicy()  // backpressure on caller
);
```

Thread Pool Sizing Rules

```
CPU-bound tasks:  coreSize = Runtime.getRuntime().availableProcessors()
I/O-bound tasks:  coreSize = cores × (1 + waitTime/cpuTime)
             or:  coreSize = cores × 2  (rough rule)
```

Rejection Policies

Policy	Behavior	Use When
`AbortPolicy`	Throws `RejectedExecutionException`	Fail fast
`CallerRunsPolicy`	Caller thread runs the task	Backpressure
`DiscardPolicy`	Silently drops task	Fire-and-forget
`DiscardOldestPolicy`	Drops oldest queued task	Best-effort

Execution Flow — Task Lifecycle

```
  submit(task)
      │
      ▼
```

```
┌──────────────────────────────────────────────────────────────┐
│                   ThreadPoolExecutor                         │
│                                                              │
│  Step 1 — active threads < corePoolSize?                     │
│           YES ──► spawn new core thread, run task immediately│
│           NO  ──► go to Step 2                               │
│                                                              │
│  Step 2 — queue has space?  (ArrayBlockingQueue capacity=200)│
│           YES ──► enqueue task, existing thread picks it up  │
│           NO  ──► go to Step 3                               │
│                                                              │
│  Step 3 — active threads < maximumPoolSize?                  │
│           YES ──► spawn surplus thread, run task immediately │
│           NO  ──► go to Step 4  (rejection)                  │
│                                                              │
│  Step 4 — RejectionHandler fires                             │
│    AbortPolicy      ──► throw RejectedExecutionException     │
│    CallerRunsPolicy ──► caller thread runs the task (slowdown)│
│    DiscardPolicy    ──► task silently dropped                │
│    DiscardOldestPolicy ──► oldest queued task dropped, retry │
└──────────────────────────────────────────────────────────────┘

Normal task lifecycle (queue has space, core thread free):

submit(task)
    │
    ▼
ArrayBlockingQueue [task1 | task2 | task3 | ...]
    │
    ▼  core thread polls
Thread-3 executes task
    │
    ▼  task done
Thread-3 returns to pool ──► polls next task from queue

Under spike (core=10, max=50, queue=200):

Requests: 0──►10  core threads spin up
Requests: 10──►210  tasks queue (queue fills)
Requests: 210──►250  surplus threads spin up (10→50)
Requests: >250  RejectionHandler fires ◄── backpressure point
```

Lifecycle — Always Shut Down Properly

```
executor.shutdown();                              // stop accepting new tasks
if (!executor.awaitTermination(30, TimeUnit.SECONDS)) {
    executor.shutdownNow();                       // interrupt running tasks
}
```

Interview One-Liner

> "Never use newCachedThreadPool in production — it creates threads without bound and can OOM. Always use ThreadPoolExecutor with a bounded queue and a rejection policy that gives backpressure. Size CPU-bound pools at core-count, I/O-bound at 2×cores."

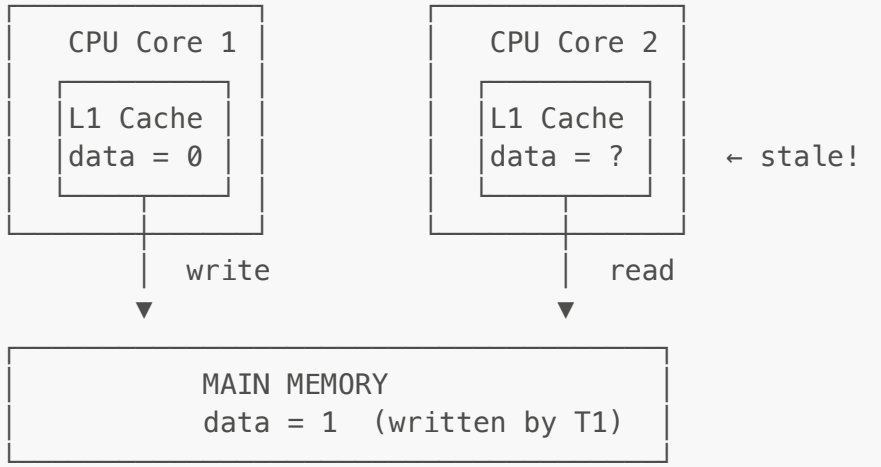
---

6. JAVA MEMORY MODEL — JMM (80% — MUST KNOW)

The Problem

JIT compilation reorders instructions for optimization, and each CPU core maintains its own L1/L2 cache that does not automatically synchronize with other cores. A write by Thread 1 may sit in its local cache indefinitely — Thread 2 can read a stale value even seconds after the write occurred.

The Core Problem: CPU Caches



T1 writes `data = 1` — sits in T1's L1 cache, never flushed to main memory. T2 reads `data` — hits T2's own L1 cache — sees stale value `0`. JMM defines which synchronization actions force a cache flush (write barrier) and a cache invalidation (read barrier).

Happens-Before Rules (visibility guarantees)

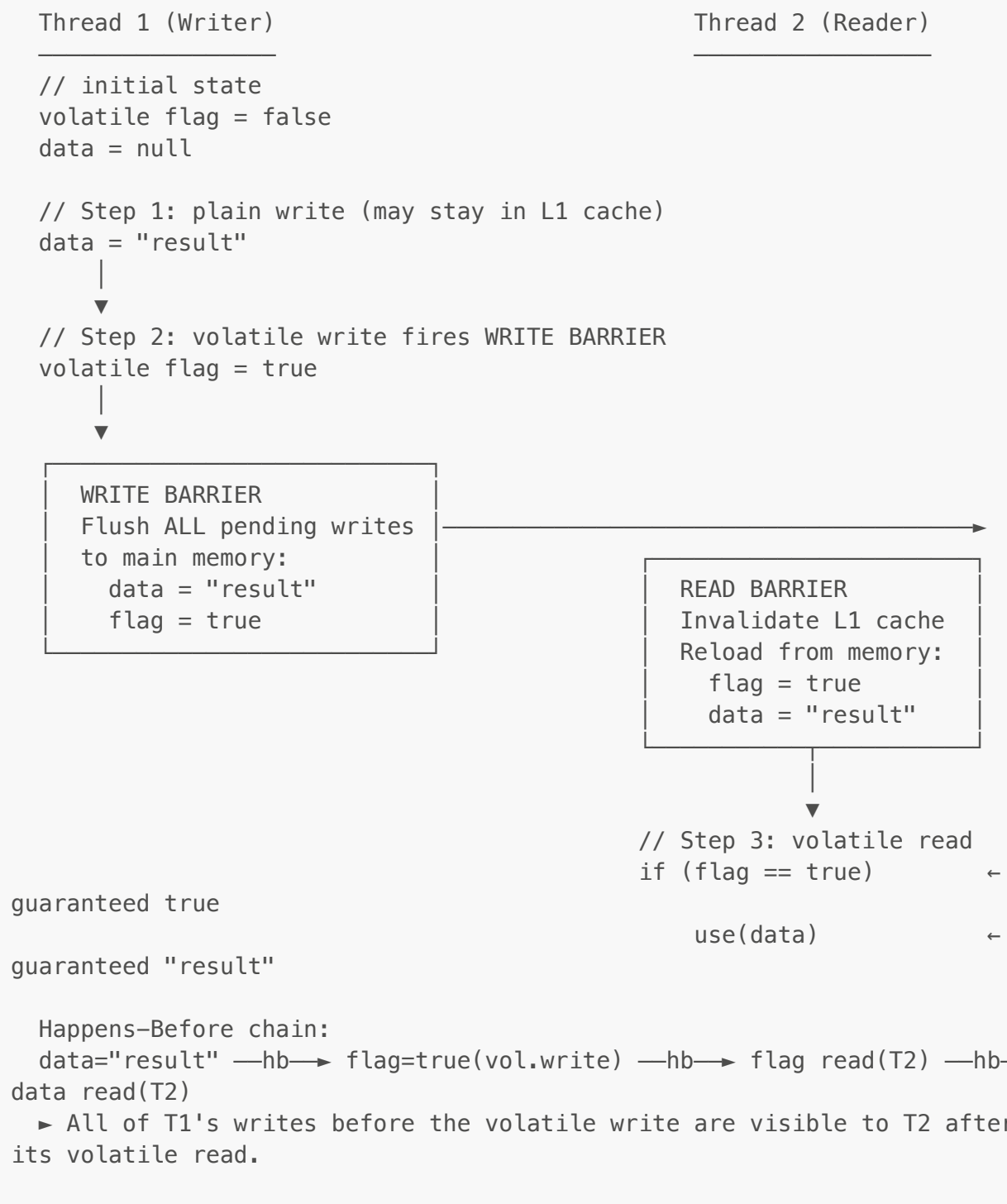
1. **Unlock** happens-before every subsequent **lock** of same monitor
2. **volatile write** happens-before every subsequent **volatile read** of same variable
3. **Thread.start()** happens-before any action in the started thread
4. **Thread.join()** — all actions in the thread happen-before join returns
5. Transitivity: if A hb B and B hb C → A hb C

Synchronized = Memory Barrier

```
// Thread 1
synchronized(lock) {
    data = "updated";            // ← WRITE BARRIER on exit: flushed to
main memory
```

```
}

// Thread 2
synchronized(lock) {              // ← READ BARRIER on enter: invalidate
cache
    System.out.println(data);     // guaranteed to see "updated"
}

// Thread 3 (NO sync)
System.out.println(data);         // MAY see stale value!
```

Execution Flow — volatile Write/Read



Double-Checked Locking — volatile required

```
// ❌ BROKEN without volatile — JIT may reorder new/init/assign
private static Singleton instance;

// ✅ CORRECT
private static volatile Singleton instance;

public static Singleton getInstance() {
    if (instance == null) {
        synchronized(Singleton.class) {
            if (instance == null) {
                instance = new Singleton();  // volatile prevents partial-
init visibility
            }
        }
    }
    return instance;
}
```

Interview One-Liner

> "JMM defines happens-before: if A hb B, then B sees all of A's writes. Entering a synchronized block is a read barrier; exiting is a write barrier. volatile provides happens-before for every write/read pair. Without these, you have no visibility guarantees — even if the value was written."

---

7. WAIT / NOTIFY — PRODUCER-CONSUMER (78% — MUST KNOW)

The Problem

Multiple threads sharing a bounded buffer must coordinate on its state: a producer must pause when the buffer is full and a consumer must pause when it is empty. Without explicit coordination, threads either corrupt shared state through races or spin wastefully burning CPU with no progress.

What Happens Without It

```
// Busy-wait anti-pattern — no wait/notify
while (queue.isEmpty()) { }   // ← spins 100% of one CPU core
T.take();

4 consumer threads all spin-waiting on an empty queue:
┌──────────┐  ┌──────────┐  ┌──────────┐  ┌──────────┐
│Consumer 1│  │Consumer 2│  │Consumer 3│  │Consumer 4│
│ 100% CPU │  │ 100% CPU │  │ 100% CPU │  │ 100% CPU │
└──────────┘  └──────────┘  └──────────┘  └──────────┘
     ▲             ▲             ▲             ▲
     └─────────────┴─────────────┴─────────────┘
```

```
                    All 4 cores saturated
                    Producer never gets CPU → queue stays empty → livelock
```

Why wait/notify

`wait()` atomically releases the monitor lock AND suspends the calling thread — the producer can now acquire the lock and add items. `notify()` / `notifyAll()` are cheap signals that only wake waiting threads when the condition they care about may have changed. No CPU burned while waiting.

Rules

```
1. Must be inside synchronized block/method
2. wait() RELEASES the lock and blocks the thread
3. notify() wakes ONE waiting thread (notifyAll() wakes all)
4. Always use while() loop — not if() — for spurious wakeup protection
```

Classic Producer-Consumer

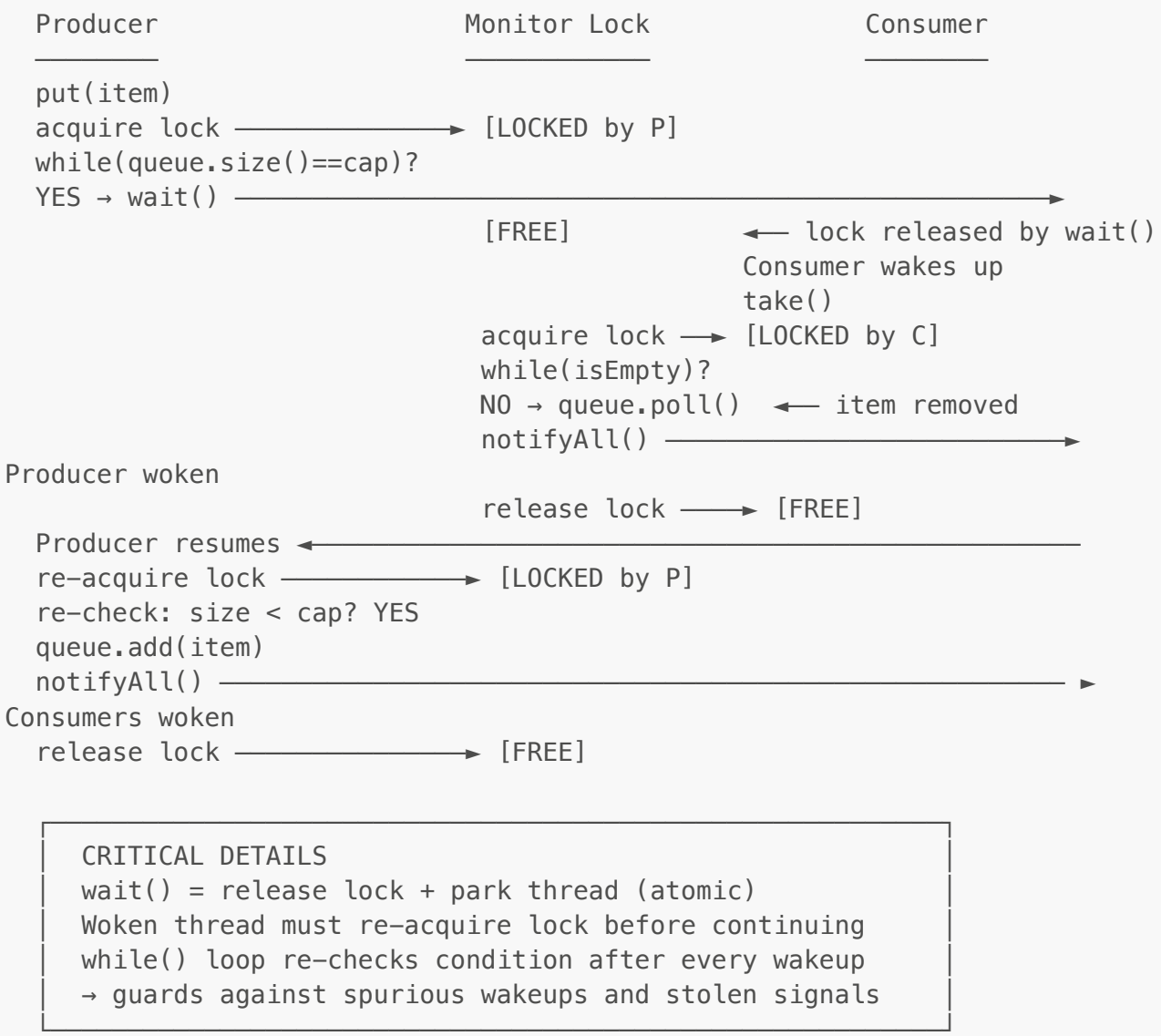
```
class BoundedBuffer<T> {
    private final Queue<T> queue = new LinkedList<>();
    private final int capacity;

    BoundedBuffer(int capacity) { this.capacity = capacity; }

    public synchronized void put(T item) throws InterruptedException {
        while (queue.size() == capacity) wait();  // wait while FULL
        queue.add(item);
        notifyAll();                              // wake consumers
    }

    public synchronized T take() throws InterruptedException {
        while (queue.isEmpty()) wait();           // wait while EMPTY
        T item = queue.poll();
        notifyAll();                              // wake producers
        return item;
    }
}
```

Why while() not if()

```
// ❌ WRONG — spurious wakeup can exit wait without condition being true
if (queue.isEmpty()) {
    wait();
    take();  // may still be empty!
}
```

```
// ✅ CORRECT — re-check condition after every wakeup
while (queue.isEmpty()) {
    wait();
}
// guaranteed non-empty here
```

Execution Flow — Producer-Consumer



Modern Alternative: BlockingQueue

```
// Prefer BlockingQueue in real code — same semantics, no manual sync
BlockingQueue<Task> queue = new LinkedBlockingQueue<>(100);

// Producer
queue.put(task);     // blocks if full
```

```
// Consumer
Task t = queue.take(); // blocks if empty
```

Interview One-Liner

> "wait/notify is the low-level mechanism for thread communication. wait() releases the lock, notify() signals. Always use while loop because of spurious wakeups. In production, prefer BlockingQueue — it encapsulates all of this correctly."

---

8. STRUCTURED CONCURRENCY — Java 25 GA (70% — SENIOR SIGNAL)

Problem — What Problem Does This Solve?

Unstructured concurrency with `ExecutorService` + `Future` creates "thread leaks" — if one subtask fails, sibling tasks keep running indefinitely with no automatic cancellation. There is no structural guarantee that all child tasks finish before the parent exits, making error propagation and resource cleanup unreliable.

What Happens Without It

```
Service launches 2 tasks: fetchUser (50ms) and fetchOrders (may fail at
t=100ms)

t=100ms: fetchOrders throws → main thread catches, returns error response
         fetchUser VT is STILL RUNNING — leaked, consuming memory/CPU

At 1,000 req/sec with 10% failure rate:
  100 leaked threads/sec accumulate
  After 5 minutes → 30,000 orphaned threads → OOM or resource exhaustion
  Exceptions from subtasks are also silently lost if Future.get() is never
called
```

Why We Need It

Structured Concurrency applies the discipline of structured programming (every block has one entry/one exit) to concurrency. Just as a method call must return before the caller continues, a scope must complete all child tasks before it exits — guaranteeing no leaks, and ensuring all exceptions surface to the owner thread.

Problem with Unstructured Concurrency

```
// ❌ 3 problems if fetchOrders() fails:
Future<User>   u = pool.submit(() -> fetchUser(id));
Future<Orders> o = pool.submit(() -> fetchOrders(id));
User user     = u.get();   // If fetchOrders fails here, fetchUser keeps
```

```
running (leak!)
Orders orders = o.get();
```

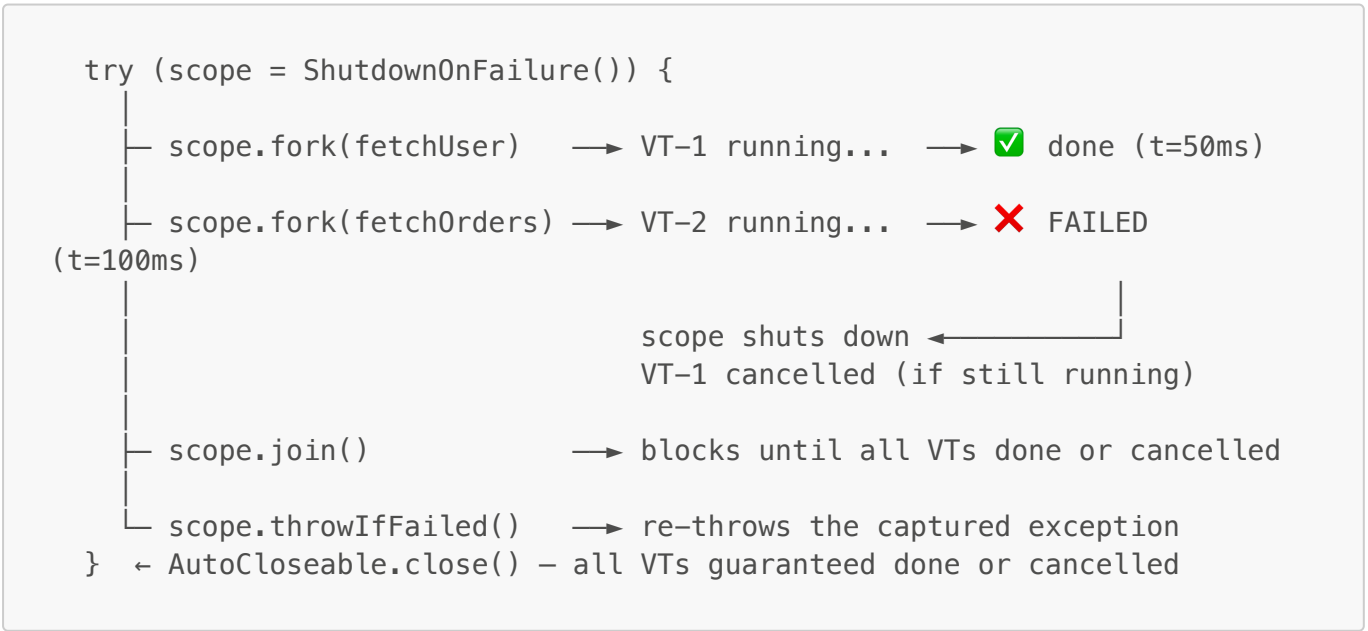
StructuredTaskScope — ShutdownOnFailure

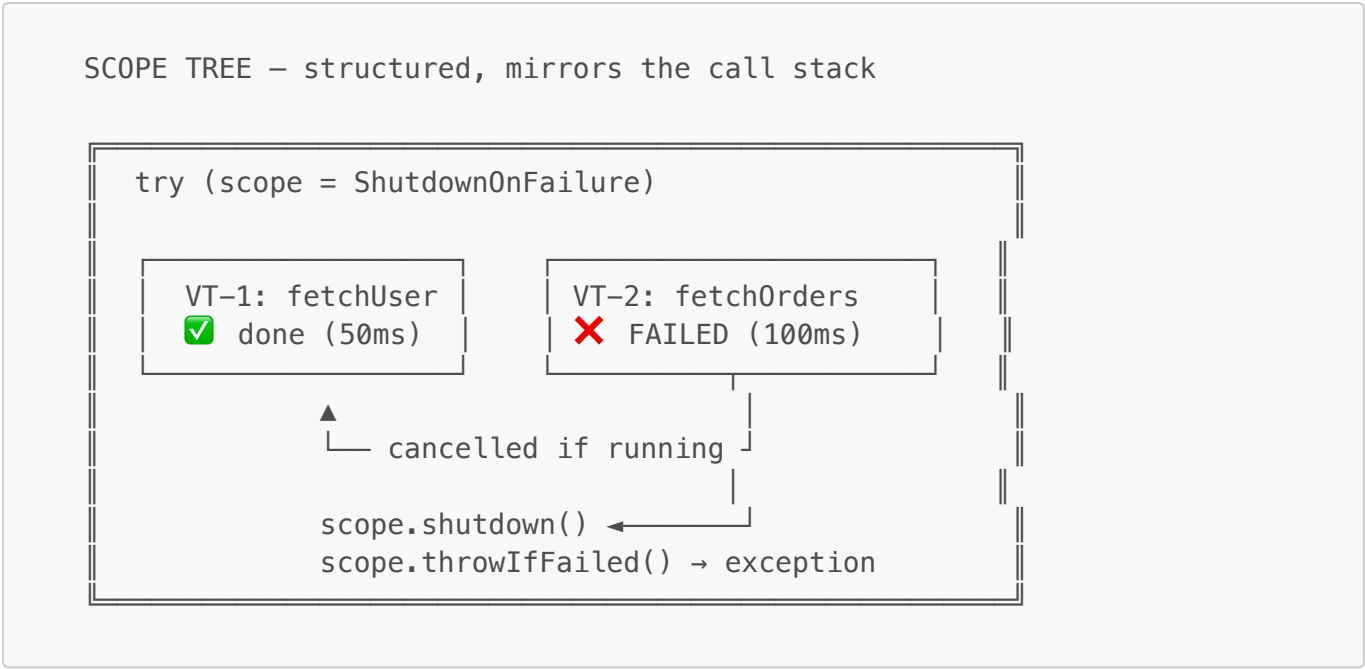
```
try (var scope = new StructuredTaskScope.ShutdownOnFailure()) {
    var userTask   = scope.fork(() -> fetchUser(id));
    var ordersTask = scope.fork(() -> fetchOrders(id));

    scope.join();           // wait for both
    scope.throwIfFailed();  // re-throw if any subtask failed

    return new Response(userTask.get(), ordersTask.get());
}
// scope closes → any still-running forks are cancelled automatically
```

ShutdownOnSuccess (first wins)

```
try (var scope = new StructuredTaskScope.ShutdownOnSuccess<String>()) {
    scope.fork(() -> queryPrimary());
    scope.fork(() -> queryReplica());

    scope.join();
    return scope.result();  // returns whichever completed first
}
```

Execution Flow





Possible Issues

```
1. scope.join() NOT called before throwIfFailed()
   → throws IllegalStateException: "Owner did not join"
   → Rule: ALWAYS call join() before throwIfFailed() or result()

2. Nested scopes: inner scope failure does NOT cancel outer scope
   try (outer = ShutdownOnFailure) {
     outer.fork(() -> {
       try (inner = ShutdownOnFailure) { /* inner fails */ }
       // inner failure is local — outer.fork's VT must re-throw to
propagate
     });
   }

3. Cancellation is COOPERATIVE — tasks must respond to interrupt
   scope.fork(() -> {
     while (true) { doCompute(); }   // ← never checks interrupt → never
cancels!
   });
   // Fix: use blocking ops (sleep/IO) which throw InterruptedException,
   //      or check Thread.currentThread().isInterrupted() in tight loops

4. ShutdownOnFailure vs ShutdownOnSuccess semantics:
   ShutdownOnFailure → ALL must succeed  (fetch user + fetch orders — need
both)
   ShutdownOnSuccess → FIRST wins        (primary DB + replica — use
whichever responds)
```

Interview One-Liner

> "Structured Concurrency (Java 25 GA) ensures child tasks never outlive their parent scope — like structured control flow for concurrency. ShutdownOnFailure cancels all siblings if one fails.

> Eliminates the resource-leak and exception-loss problems of raw ExecutorService + Future."

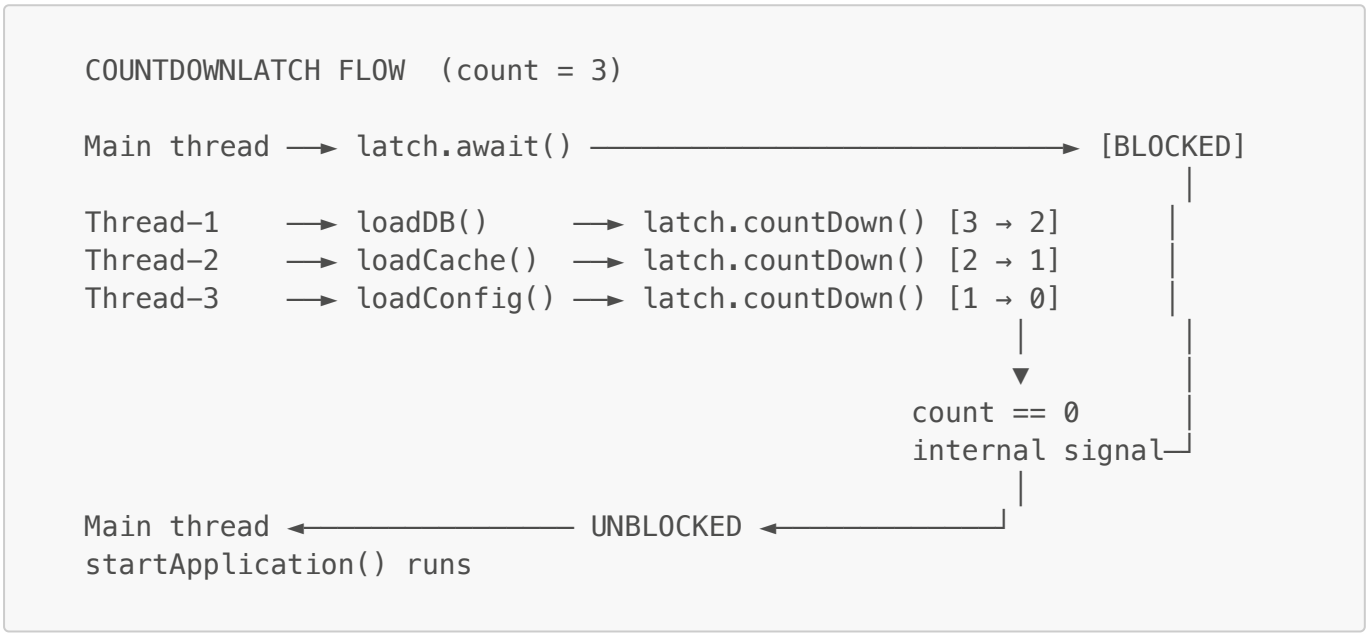
---

9. CONCURRENCY UTILITIES (68% — SHOULD KNOW)

CountDownLatch — one-time gate

Problem: N initialization tasks (load DB, load cache, load config) must all complete before the application serves traffic. Raw `Thread.join()` can only wait on one thread at a time and is not composable.

What happens without it: Services start with partial initialization — NullPointerExceptions on first requests, failed health checks, incorrect state before dependencies are ready.

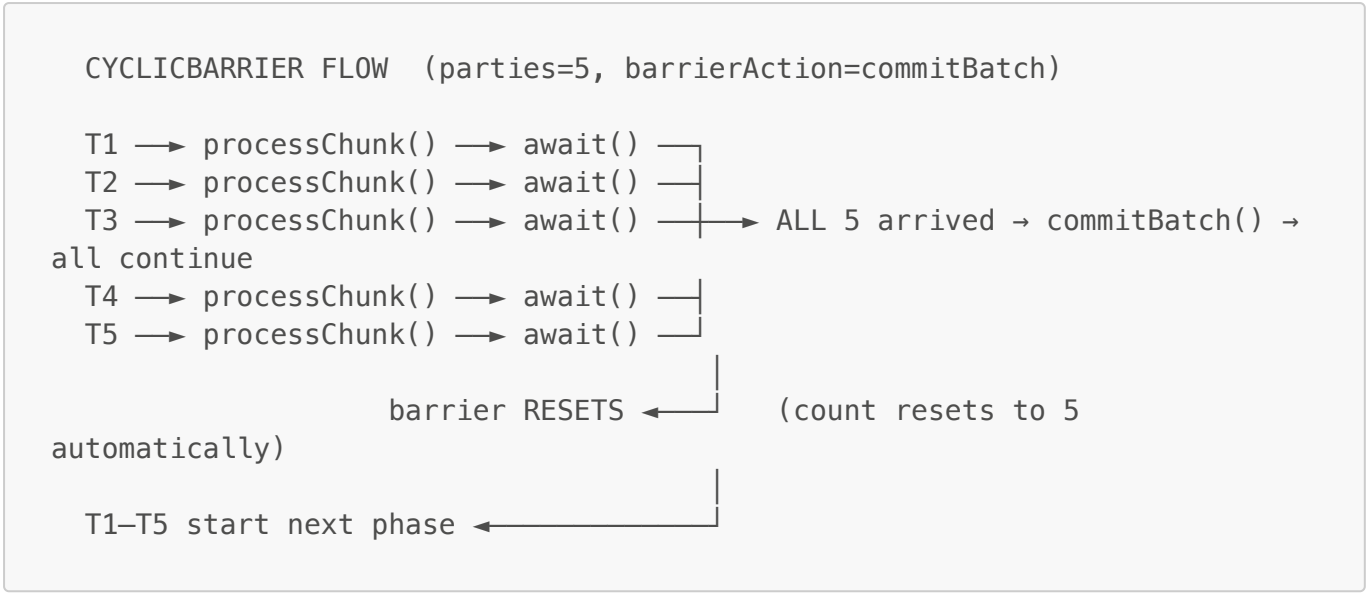



```
CountDownLatch latch = new CountDownLatch(3);  // count = 3

executor.submit(() -> { loadDB();     latch.countDown(); });
executor.submit(() -> { loadCache();  latch.countDown(); });
executor.submit(() -> { loadConfig(); latch.countDown(); });

latch.await();              // blocks until count reaches 0
startApplication();         // runs only after all 3 complete

// Key: NOT reusable. Count can only go down, not reset.
```

CyclicBarrier — reusable, all-threads-sync

Problem: N worker threads process data in phases and must all complete phase K before any thread starts phase K+1. Without coordination, fast threads race ahead on stale partial data written by slow threads.

What happens without it: Thread-1 finishes phase 1, begins phase 2, reads intermediate values Thread-3 has not yet written — corrupt merge results, wrong aggregations.

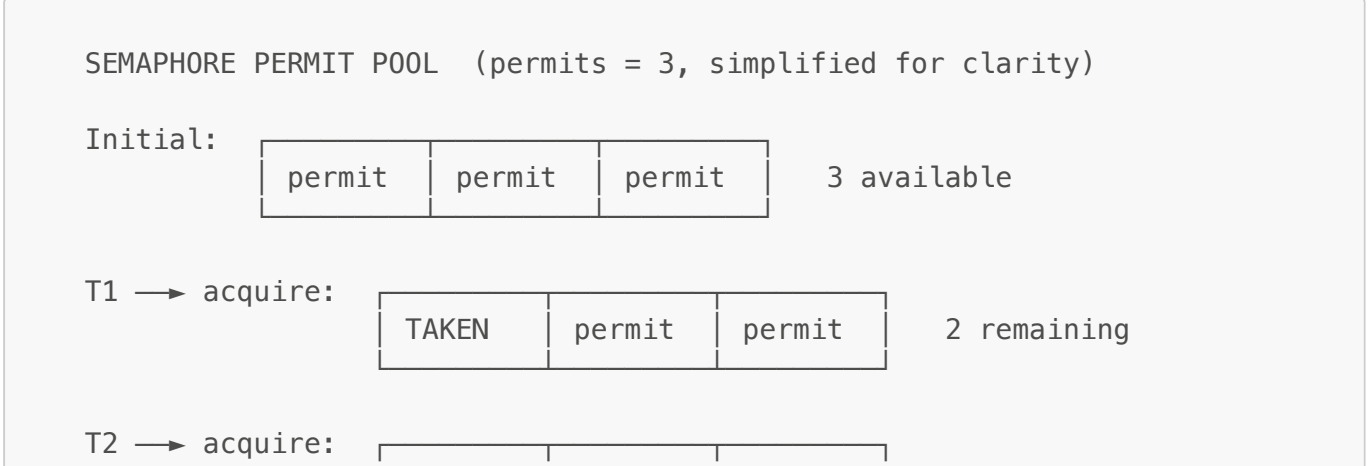


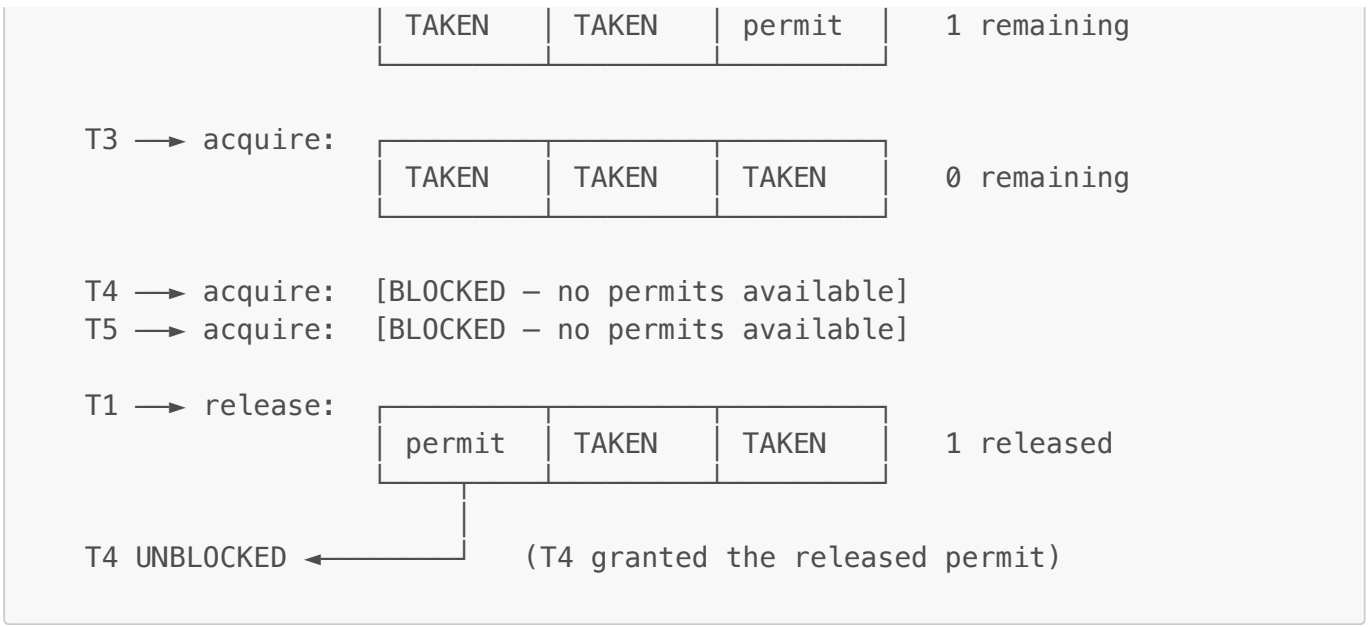

```
CyclicBarrier barrier = new CyclicBarrier(5, () -> commitBatch());  // action on trip

for (int i = 0; i < 5; i++) {
    new Thread(() -> {
        for (int phase = 0; phase < 10; phase++) {
            processChunk();
            barrier.await();   // all 5 must arrive before any continue
        }
    }).start();
}

// Key: Reusable — resets after all threads arrive. Good for multi-phase batch jobs.
```

Semaphore — rate limiting / resource pool

Problem: An external payment gateway allows only 10 concurrent connections. Without throttling, 200 pool threads all call it simultaneously — 429 Too Many Requests, cascading failures for all 200.

What happens without it: All 200 threads hit the rate limit simultaneously, the gateway trips its circuit breaker, ALL requests fail rather than just the 190 excess ones.






```
Semaphore semaphore = new Semaphore(10);  // max 10 concurrent

executor.submit(() -> {
    semaphore.acquire();          // blocks if all 10 permits taken
    try {
        callPaymentGateway();     // max 10 concurrent calls
    } finally {
        semaphore.release();
    }
});
```

Phaser — flexible barrier (dynamic parties)

```
Phaser phaser = new Phaser(1);   // register main thread

for (int i = 0; i < 5; i++) {
    phaser.register();           // dynamically add party
    new Thread(() -> {
        doWork();
        phaser.arriveAndAwaitAdvance();  // advance phase
        doMoreWork();
        phaser.arriveAndDeregister();    // leave phaser
    }).start();
}

phaser.arriveAndDeregister();    // main thread done
```

Decision Guide

Need	Use
Wait for N tasks to complete once	`CountDownLatch`
Sync N threads repeatedly at checkpoints	`CyclicBarrier`
Limit concurrent access to N	`Semaphore`
Dynamic party count, multi-phase	`Phaser`
Single-thread ordered execution	`SingleThreadExecutor`

---

10. FORK/JOIN FRAMEWORK (60% — SHOULD KNOW)

The Problem

Sorting 1 billion integers single-threaded is O(n log n) but uses only 1 of 8 CPU cores — the other 7 sit idle. Divide-and-conquer algorithms are naturally recursive and splittable, but `ExecutorService` cannot express recursive task decomposition or redistribute capacity from idle threads to busy ones dynamically.

What Happens Without It

```
Single-threaded merge sort of 1 billion integers:
  1 core active, 7 cores idle → ~30 seconds wall time
  CPU utilization: 12.5%

ExecutorService with manual pre-split into 8 tasks:
  Works only if data is perfectly uniform
  No work stealing — Thread-0 finishes in 2s, Thread-7 finishes in 8s
  Threads 0–6 sit idle for 6 seconds waiting at the join point
  Uneven data distribution → stragglers stall all threads
```

Key Idea: Divide & Conquer + Work Stealing

```
class MergeSort extends RecursiveAction {
    private final int[] arr;
    private final int from, to;

    @Override
    protected void compute() {
        if (to - from <= 1000) {
            Arrays.sort(arr, from, to);   // base case — do sequentially
            return;
        }
        int mid = (from + to) / 2;
        MergeSort left  = new MergeSort(arr, from, mid);
        MergeSort right = new MergeSort(arr, mid, to);
        invokeAll(left, right);           // fork both, join both
        merge(arr, from, mid, to);
    }
```

```
}

ForkJoinPool pool = ForkJoinPool.commonPool();
pool.invoke(new MergeSort(arr, 0, arr.length));
```

Work Stealing

```
  Each thread maintains its own DEQUE of tasks.
  Owner pushes/pops from the LEFT end (LIFO — recently forked tasks reuse
warm cache).
  Idle thieves steal from the RIGHT end of busy threads' deques (FIFO).
  Owner and thief work opposite ends → minimal lock contention on the
deque.


──────────────────────────────────────────────────────────────────────────

  Thread-0 deque:  ◄─LIFO── [task-A] [task-B] [task-C] [task-D] ──FIFO─►
                    T0 pops ─┘                                  └─ steal
target

  Thread-1 deque:  []  ← idle!

──────────────────────────────────────────────────────────────────────────
                                                        │
                          STEAL ◄──────────────────────┘
                          Thread-1 steals task-D from RIGHT end of T0's
deque

  Thread-0 deque:  ◄─LIFO── [task-A] [task-B] [task-C]  (task-D gone)
  Thread-1 deque:  [task-D]  ← now working in parallel

──────────────────────────────────────────────────────────────────────────

  Both threads busy. Contention only arises if T0's deque has exactly 1
item.
```

Execution Flow — RecursiveAction.compute()

```
  pool.invoke(sort(0..1B))
        │
        ▼
  compute(0..1B)  [Thread-0]
    ├─ fork left(0..500M)    ──► new ForkJoinTask queued on T0's deque
    ├─ fork right(500M..1B)  ──► new ForkJoinTask queued on T0's deque
    │                                        │
    │              Thread-1 idle ────────────┘  steals right(500M..1B)
    │
    ├─ invokeAll(left, right) ← T0 works on left
    │                           right stolen and run by T1 concurrently
    │
```

```
    ├─ T0: compute(0..500M)            T1: compute(500M..1B)
    │      ├─ fork(0..250M)                ├─ fork(500M..750M)
    │      ├─ fork(250M..500M)             └─ fork(750M..1B)
    │      └─ recurse → base case          └─ recurse → base case
    │                                           (stolen by T2, T3, ...)
    └─ merge results bottom-up as subtasks complete
```

parallelStream uses ForkJoinPool.commonPool()

```
// Uses ForkJoinPool.commonPool() — shared across entire JVM!
list.parallelStream()
    .filter(item -> item.isValid())
    .map(this::process)
    .collect(toList());

// Isolate from common pool to avoid starving other parallelStreams
ForkJoinPool custom = new ForkJoinPool(4);
custom.submit(() -> list.parallelStream().forEach(this::process)).get();
```

Interview One-Liner

> "ForkJoin is best for CPU-intensive divide-and-conquer. Work stealing keeps all threads busy. Be careful with parallelStream — it uses the shared commonPool, so long-running tasks starve other callers. Use a custom ForkJoinPool when isolation matters."

---

11. THREADLOCAL (58% — SHOULD KNOW)

The Problem

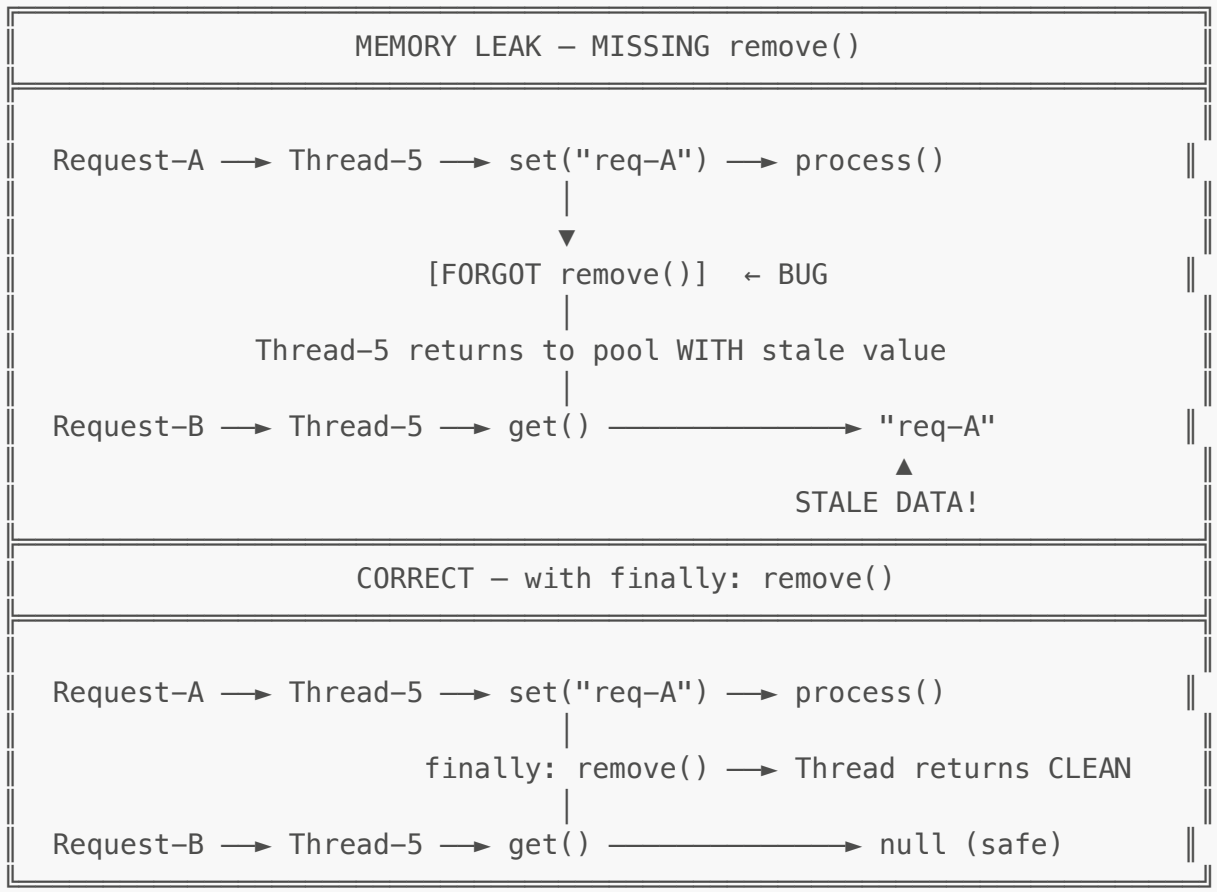
HTTP requests carry context — traceId, userId, locale — needed deep inside the call stack (service → repository → utility → logger). Passing this as a parameter through every layer pollutes every method signature and couples unrelated classes to request metadata.

What Happens Without It

```
Option A — parameter pollution:
  processOrder(orderId, traceId, userId, locale)
    → validatePayment(payment, traceId, userId, locale)
      → auditLog(event, traceId, userId, locale)   // every method carries
it

Option B — shared ConcurrentHashMap<Thread, Context>:
  Read:  map.get(Thread.currentThread())   // sync overhead on every access
  Risk:  map.remove() missed → memory leak proportional to thread-pool size
```

ThreadLocal gives per-thread storage with zero synchronization — each thread has its own copy, isolated by definition.

Memory Leak Scenario — Thread Pool Reuse



Use Case: Per-Request Context (no parameter passing)

```
public class RequestContext {
    private static final ThreadLocal<String> requestId = new ThreadLocal<>
();

    public static void set(String id)   { requestId.set(id); }
    public static String get()          { return requestId.get(); }
    public static void clear()          { requestId.remove(); }  //
CRITICAL
}

// Spring Filter
@Override
protected void doFilterInternal(HttpServletRequest req, ...) {
    RequestContext.set(UUID.randomUUID().toString());
    try {
        filterChain.doFilter(req, response);
    } finally {
        RequestContext.clear();  // always clear or memory leak in thread
```

```
pools!
    }
}
```

Memory Leak Pattern in Thread Pools

```
Thread pool threads are REUSED. ThreadLocal values survive across requests
if you don't call remove(). Next request on same thread sees previous
request's data.

Rule: If using ThreadLocal with a thread pool, ALWAYS remove() in a finally
block.
```

InheritableThreadLocal — pass to child threads

```
InheritableThreadLocal<String> ctx = new InheritableThreadLocal<>();
ctx.set("user-123");

new Thread(() -> {
    System.out.println(ctx.get());  // prints "user-123"
}).start();

// Note: does NOT propagate to virtual threads — use ScopedValue instead
```

Possible Issues / Gotchas

- **Thread pool reuse**: ThreadLocal values outlive the request if `remove()` is not called in a `finally` block — the most common production bug.
- **InheritableThreadLocal + thread pools**: Child threads in a pool inherit the value at pool creation time, not task submission time — values are stale.
- **Virtual threads**: 1 million virtual threads × ThreadLocal copy = heap explosion. Use `ScopedValue` instead.
- **ClassLoader leaks**: Web container undeploy + ThreadLocal holding a class from the webapp classloader prevents GC of the entire classloader.

Interview One-Liner

> "ThreadLocal stores per-thread data — great for request IDs and Spring Security's SecurityContextHolder. Critical pitfall: always call remove() in thread pools or you get stale data and memory leaks. For virtual threads, use ScopedValue (Java 21+) — it's immutable and scope-bound."

---

12. THREAD FUNDAMENTALS (55% — GOOD TO KNOW)

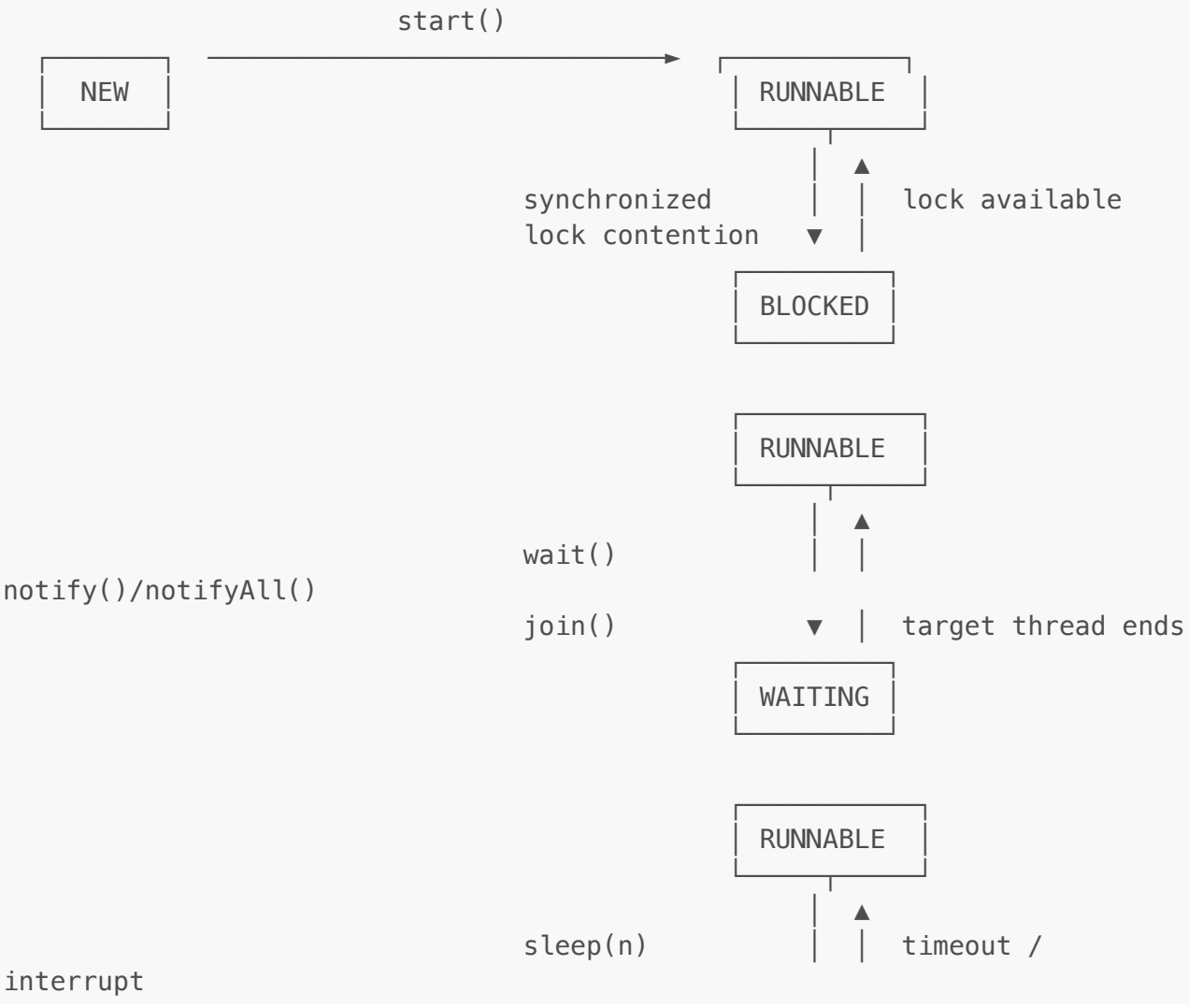
The Problem

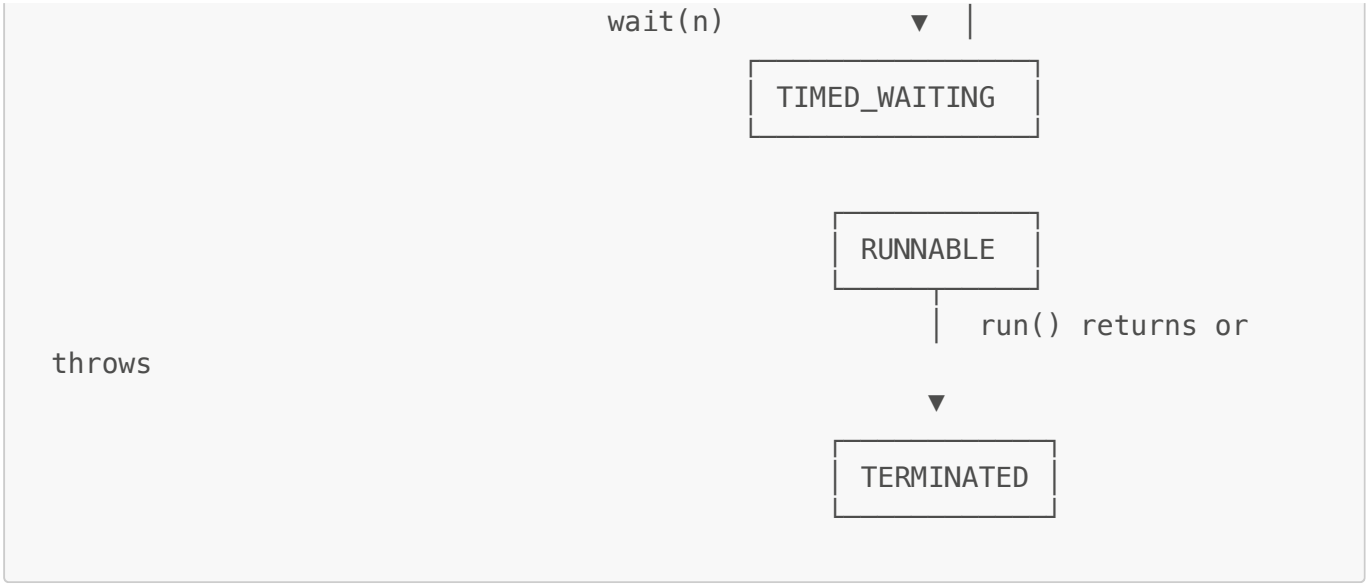
A single CPU core executes one instruction at a time. Without concurrency, a web server handling a request that waits 200ms for a DB response leaves the CPU 100% idle. Every other user queues behind it. Threads let us interleave work so the CPU stays busy while one task waits on I/O.

Why Threads (not processes)

```
┌──────────────────────────────────────────────────────┐
│                  PROCESS vs THREAD                   │
├──────────────────────────┬───────────────────────────┤
│        Process           │          Thread           │
├──────────────────────────┼───────────────────────────┤
│  Separate memory space   │  Shared heap within process │
│  Separate file handles   │  Shared file handles      │
│  Context switch ~10µs    │  Context switch ~1µs      │
│  IPC needed to share     │  Direct read/write shared data │
│  OS-level isolation      │  JVM-level management     │
└──────────────────────────┴───────────────────────────┘
Threads are cheap to create and share memory — ideal for
concurrent I/O within a single service process.
```

Thread State Machine





start() vs run()

```
thread.run();    // executes in CURRENT thread — no new thread created!
thread.start();  // creates NEW thread → JVM calls run() in it

// ❌ thread.run() → both print "main"
// ✅ thread.start() → prints "Thread-0"
```

Thread States

```
NEW → RUNNABLE → { BLOCKED | WAITING | TIMED_WAITING } → TERMINATED

BLOCKED:       waiting to acquire a synchronized lock
WAITING:       wait(), join() with no timeout
TIMED_WAITING: sleep(n), wait(n), join(n)
```

Daemon Threads

```
thread.setDaemon(true);   // must be set BEFORE start()
thread.start();

// JVM exits when ONLY daemon threads remain
// GC, JIT compiler, finalizer are all daemon threads
// Default: non-daemon (JVM waits for them)
```

join() internals

```
// join() internally calls wait() on the thread object
// → releases the lock, blocks caller until target thread terminates
```

```
t1.join();   // current thread waits; t1's completion notifies it via
notifyAll()
```

Thread Lifecycle — IllegalThreadStateException

```
thread.start();
thread.start();  // ❌ throws IllegalThreadStateException
                 // threadStatus is no longer NEW (0)
                 // Create a new Thread object to rerun the task
```

Possible Issues / Gotchas

- **`run()` instead of `start()`**: Runs synchronously on the calling thread. No new thread is spawned — the most common beginner mistake.
- **Double `start()`**: A Thread object cannot be restarted. Create a new instance.
- **Daemon thread data loss**: JVM kills daemon threads abruptly on exit. Never do durable I/O on daemon threads.
- **`Thread.interrupt()`**: Sets the interrupted flag. Blocking calls (`sleep`, `wait`, `join`) throw `InterruptedException` and CLEAR the flag. Always re-interrupt or propagate — never swallow silently.

---

13. SPRING @ASYNC + MDC PROPAGATION (55% — GOOD TO KNOW)

The Problem

The Tomcat HTTP thread that receives a request also executes all downstream work — including slow operations like sending emails (200ms), writing audit logs, or calling third-party webhooks. The user's browser waits for all of it before getting an HTTP response.

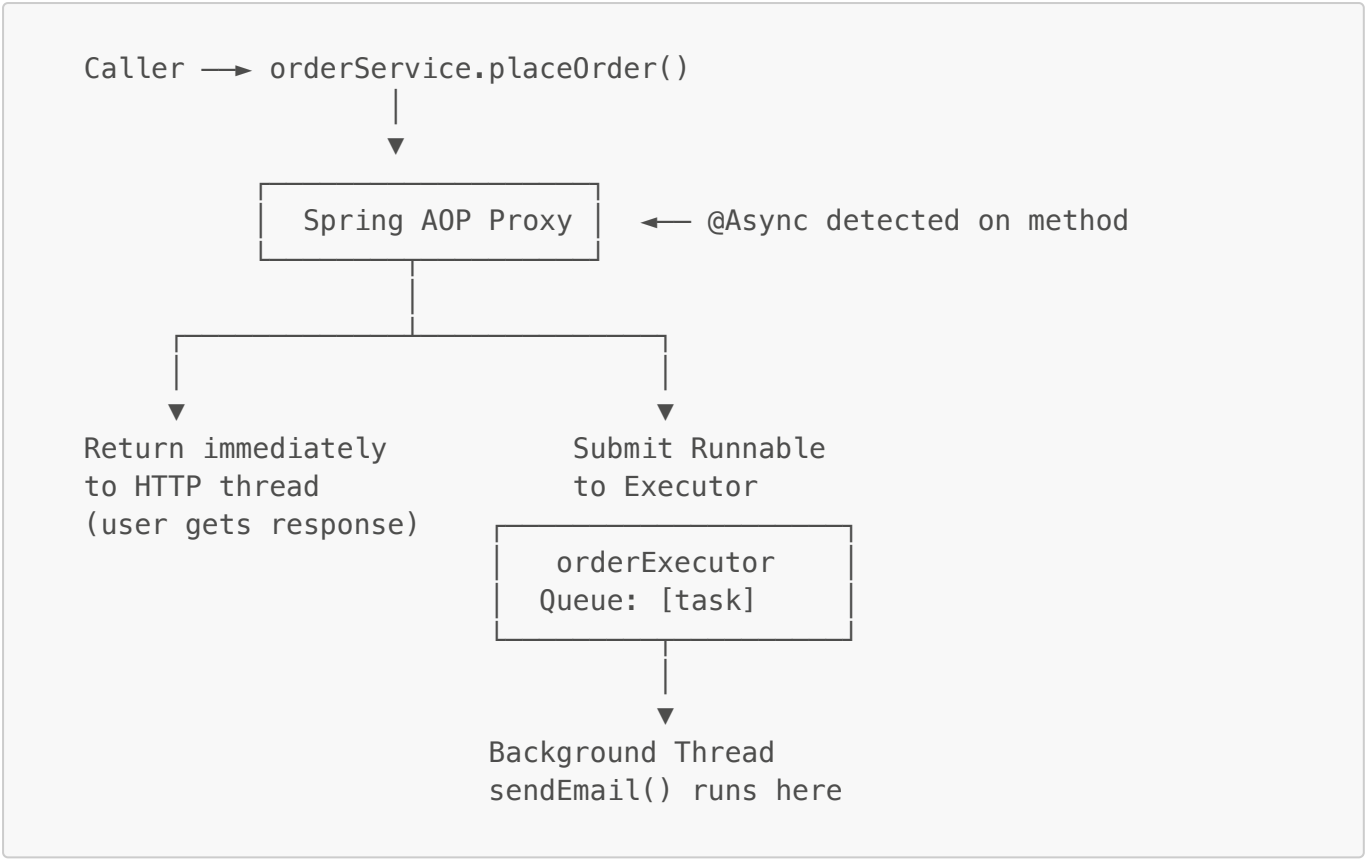
What Happens Without @Async

```
200ms email × 500 concurrent requests = Tomcat pool exhausted

  Tomcat default pool = 200 threads
  500 requests × 200ms email = all 200 threads blocked in email client
  → 300 requests queue up → response time = email latency + queue wait
  → Under spike: connection timeout for end users
```

Why @Async

Return HTTP 200 immediately. Hand the slow work to a background executor pool sized independently from the Tomcat pool. The user gets instant feedback; email delivery happens in parallel.

AOP Proxy Interception — How @Async Actually Works



Execution Flow

```
Step 1:  HTTP request arrives → Tomcat assigns Thread-12
Step 2:  Thread-12 calls orderService.placeOrder()
Step 3:  Spring AOP proxy intercepts (because @Async present)
Step 4:  Proxy wraps sendEmail() body as a Runnable
Step 5:  Proxy submits Runnable to orderExecutor's BlockingQueue
Step 6:  Proxy returns immediately to Thread-12
Step 7:  Thread-12 returns HTTP 200 to user ← user sees response NOW
Step 8:  orderExecutor worker picks up Runnable from queue
Step 9:  sendEmail() executes on order-async-3 (background thread)
```

How @Async Works

```
@Service
public class NotificationService {
    @Async                                      // Spring creates AOP proxy
    public CompletableFuture<Void> sendEmail(String email) {
        emailClient.send(email);                // runs in background
executor
        return CompletableFuture.completedFuture(null);
    }
}

// ❌ GOTCHA: self-invocation bypasses proxy
@Service
public class OrderService {
```

```
    @Autowired NotificationService self;

    public void placeOrder() {
        self.sendEmail("...");    // ✅ via injected proxy — async
        sendEmail("...");         // ❌ direct call — runs synchronously!
    }
}
```

Custom Thread Pool for @Async

```
@Configuration
@EnableAsync
public class AsyncConfig {
    @Bean("orderExecutor")
    public Executor orderExecutor() {
        ThreadPoolTaskExecutor exec = new ThreadPoolTaskExecutor();
        exec.setCorePoolSize(10);
        exec.setMaxPoolSize(50);
        exec.setQueueCapacity(200);
        exec.setThreadNamePrefix("order-async-");
        exec.initialize();
        return exec;
    }
}

@Async("orderExecutor")   // use specific pool
public void processAsync() { ... }
```

MDC Propagation to Async Threads

```
@Bean
public Executor asyncExecutor() {
    ThreadPoolTaskExecutor exec = new ThreadPoolTaskExecutor();
    exec.setTaskDecorator(runnable -> {
        Map<String, String> mdcContext = MDC.getCopyOfContextMap();  //
capture
        return () -> {
            MDC.setContextMap(mdcContext != null ? mdcContext :
Collections.emptyMap());
            try   { runnable.run(); }
            finally { MDC.clear(); }
        };
    });
    exec.initialize();
    return exec;
}
```

Possible Issues / Gotchas

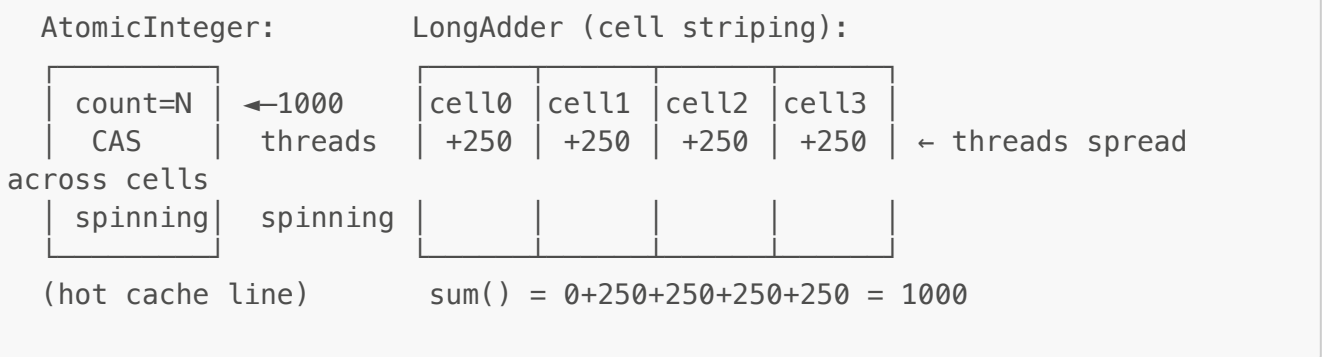
- **Self-invocation bypasses proxy**: `this.sendEmail()` never hits the AOP proxy. Inject the bean into itself or refactor to a separate bean.
- **`void` return swallows exceptions silently**: Return `CompletableFuture<Void>` and attach `.exceptionally()`, or configure `AsyncUncaughtExceptionHandler`.
- **Default executor is `SimpleAsyncTaskExecutor`**: Creates a new thread per invocation with no pooling. Always define a custom `ThreadPoolTaskExecutor`.
- **MDC context lost**: Background thread has a blank MDC map — traceId disappears from logs. Fix with `TaskDecorator` as shown above.
- **`@EnableAsync` required**: Without it, `@Async` annotations are silently ignored — methods run synchronously.

---

14. LONGADDER vs ATOMICINTEGER (45% — GOOD TO KNOW)

The Problem

AtomicInteger under high contention: 1000 threads calling `incrementAndGet()` simultaneously. CAS fails → retry → fail → retry → all 1000 threads spinning on the same memory location. This creates a hot cache line that bounces between CPU cores, destroying throughput.




```
// AtomicInteger — CAS spin loop under high contention
AtomicInteger counter = new AtomicInteger();
counter.incrementAndGet();   // fast for LOW contention

// LongAdder — cell-based striping, merges on read
LongAdder adder = new LongAdder();
adder.increment();           // fast for HIGH contention (metrics,
counters)
long total = adder.sum();    // slightly stale but fine for stats

// Rule: metrics/monitoring → LongAdder
//       compare-and-swap logic → AtomicInteger
```

Possible Issues

- `sum()` is NOT atomic: a thread incrementing between cell reads gives a slightly stale total. Fine for metrics dashboards, wrong for IDs or compare-and-set logic.

- `LongAdder` has higher memory overhead than `AtomicInteger` — it allocates a `Cell[]` array on first contention.
- `sumThenReset()` is also non-atomic — can miss increments that happen during the operation.
- No `compareAndSet()` equivalent — if you need CAS semantics, stick with `AtomicLong`.

---

15. NON-BLOCKING vs ASYNC (50% — GOOD TO KNOW)

The Problem

10,000 concurrent payment requests × 500ms payment API = need 10,000 threads simultaneously. Platform thread pool (200 threads): 9,800 requests queued, latency spikes to seconds. Each platform thread blocked on I/O wastes ~1 MB stack and one OS scheduler slot.

```
  BLOCKING (ExecutorService):        REACTIVE (WebClient):       VIRTUAL
THREADS:
  Thread-1 ──►[API 500ms BLOCKED]   Thread-1 ──► subscribe()   VT-1 ──► API
──► [PARKED]
  Thread-2 ──►[API 500ms BLOCKED]             ──► callback     carrier
picks up VT-5001
  ...200 threads all BLOCKED         1 event loop thread       carrier
picks up VT-2...
  9800 requests QUEUED               handles 10k concurrently  1M VTs, 8
carrier threads
```

Aspect	`ExecutorService` + `Future`	WebClient (Reactor/Netty)	Virtual Threads
Main thread freed?	✅	✅	✅
Pool thread blocks?	✅ Yes	❌ No	❌ (JVM parks)
Max concurrency	Pool size	1000s	Millions
Code style	Imperative	Functional/reactive	Imperative
Backpressure	Manual	Built-in	None (OOM risk)
Best for	100s concurrent	1000s, streaming	I/O-heavy services

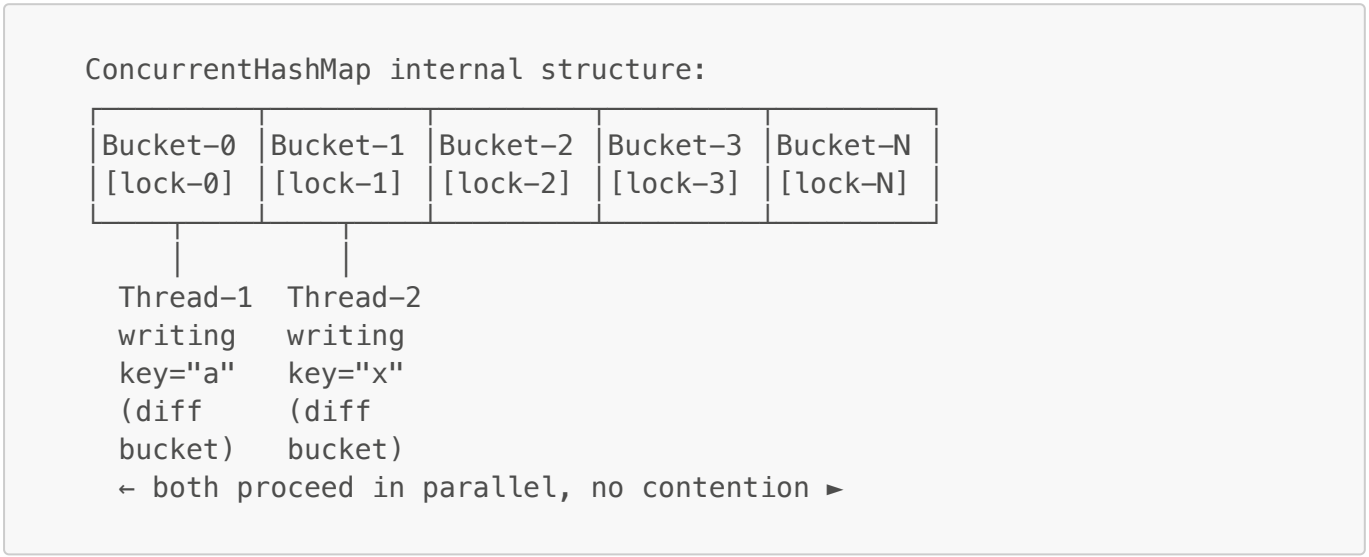
```
// WebClient — truly non-blocking
webClient.post().uri("/pay").bodyValue(req)
    .retrieve()
    .bodyToMono(PaymentResponse.class)
    .retryWhen(Retry.backoff(3, Duration.ofSeconds(1)))
    .subscribe(response -> process(response));  // no thread blocked
```

---

16. CONCURRENT COLLECTIONS (65% — SHOULD KNOW)

The Problem

`HashMap` is NOT thread-safe. Two threads resizing simultaneously → infinite loop (Java 7) or data loss (Java 8+). `Collections.synchronizedMap` wraps with a single lock — every read AND write blocks ALL other threads. Even iteration requires external synchronization on `synchronizedMap`, or you get `ConcurrentModificationException`.



ConcurrentHashMap — segment-level locking

```
Map<String, Integer> map = new ConcurrentHashMap<>();

// Atomic compound operations (no external sync needed)
map.putIfAbsent("key", 1);                      // atomic check-then-put
map.compute("key", (k, v) -> v == null ? 1 : v + 1);  // atomic update
map.merge("key", 1, Integer::sum);              // atomic merge
map.computeIfAbsent("key", k -> expensiveInit(k)); // init once

// Reads NEVER block. Writes lock only the affected bucket.
// 16 default segments → 16× concurrency vs synchronizedMap
```

ConcurrentHashMap vs synchronizedMap:

```
synchronizedMap:  single lock, every read + write blocks ALL threads
ConcurrentHashMap: bucket-level lock, reads non-blocking, 10-50× throughput
```

Execution Flow — CHM.compute()

```
map.compute("orders:user42", (k, v) -> v == null ? 1 : v + 1)

Step 1 ──► hash("orders:user42") ──► bucket index = 7
```

```
  Step 2 ──► CAS lock on bucket-7 node head
             ┌──────────────────────────────────────┐
             │ Bucket-7  [LOCKED by Thread-1]       │
             │  Node: key="orders:user42", val=5    │
             └──────────────────────────────────────┘

  Step 3 ──► lambda runs: v=5 → returns 6

  Step 4 ──► write val=6, release bucket-7 lock

  Step 5 ──► Thread-2 was waiting on bucket-7
             ──► now acquires ──► reads val=6 ──► returns 7

  All other buckets: completely unaffected, zero contention
```

CopyOnWriteArrayList — read-heavy, rare writes

```
List<String> list = new CopyOnWriteArrayList<>();

// Every write creates a NEW copy of the internal array
list.add("item");       // expensive: array copy
list.get(0);            // cheap: no lock at all (reads snapshot)

// ✅ Perfect for: event listener lists, config lists (read 1000×, write
once)
// ❌ Avoid for: high-write scenarios — copying is O(n) per write

// Iteration is SAFE — iterates over snapshot, never throws
ConcurrentModificationException
for (String s : list) { ... }  // safe even if another thread modifies list
```

BlockingQueue variants

```
// LinkedBlockingQueue — optionally bounded, FIFO
BlockingQueue<Task> q1 = new LinkedBlockingQueue<>(100);

// ArrayBlockingQueue — bounded, FIFO, fairer
BlockingQueue<Task> q2 = new ArrayBlockingQueue<>(100, true); // fair=true

// PriorityBlockingQueue — unbounded, sorted by priority
BlockingQueue<Task> q3 = new PriorityBlockingQueue<>();

// SynchronousQueue — zero capacity, direct handoff (used in
cachedThreadPool)
BlockingQueue<Task> q4 = new SynchronousQueue<>();
```

```
// DelayQueue — elements only available after delay expires
DelayQueue<DelayedTask> q5 = new DelayQueue<>();
```

Decision Table

Collection	Thread-Safe	Use When
`ConcurrentHashMap`	✅	High-concurrency key-value store
`CopyOnWriteArrayList`	✅	Many reads, rare writes (listeners)
`LinkedBlockingQueue`	✅	Producer-consumer, bounded buffer
`ConcurrentLinkedQueue`	✅	Non-blocking queue, high throughput
`Collections.synchronizedList`	✅ (coarse)	Legacy, avoid in new code

Interview One-Liner

> "ConcurrentHashMap is the go-to — concurrent reads never block, writes lock only one bucket, and compound ops like compute/merge are atomic. CopyOnWriteArrayList is for read-heavy lists where iteration safety matters. Always prefer these over synchronized wrappers."

---

17. REENTRANTLOCK + CONDITION (62% — SHOULD KNOW)

When to Use Over synchronized

```
// synchronized limitations:
// 1. No timeout — blocks forever
// 2. No interruptible lock acquisition
// 3. One implicit condition (wait/notifyAll)
// 4. Causes virtual thread PINNING on blocking I/O

// ReentrantLock fixes all four:
ReentrantLock lock = new ReentrantLock();

// tryLock with timeout — prevents deadlock
if (lock.tryLock(1, TimeUnit.SECONDS)) {
    try {
        // critical section
    } finally {
        lock.unlock();  // ALWAYS in finally
    }
} else {
    // couldn't acquire lock — handle gracefully
}

// lockInterruptibly — responds to Thread.interrupt()
lock.lockInterruptibly();
```

Multiple Conditions — precise signalling

```
ReentrantLock lock = new ReentrantLock();
Condition notFull  = lock.newCondition();  // signal producers
Condition notEmpty = lock.newCondition();  // signal consumers

class BoundedBuffer<T> {
    private final Queue<T> queue = new LinkedList<>();
    private final int capacity = 10;

    void put(T item) throws InterruptedException {
        lock.lock();
        try {
            while (queue.size() == capacity) notFull.await();  // wait for
space
            queue.add(item);
            notEmpty.signal();  // wake ONE consumer only (vs notifyAll)
        } finally { lock.unlock(); }
    }

    T take() throws InterruptedException {
        lock.lock();
        try {
            while (queue.isEmpty()) notEmpty.await();  // wait for item
            T item = queue.poll();
            notFull.signal();   // wake ONE producer only
            return item;
        } finally { lock.unlock(); }
    }
}
// Advantage over wait/notifyAll: signal() wakes only the RIGHT threads
// notifyAll() wakes everyone → extra context switches
```

Fair Lock — prevent starvation

```
ReentrantLock fairLock = new ReentrantLock(true);  // FIFO ordering
// Threads acquire in the order they requested — no thread starves
// Slight throughput cost vs unfair (default)
```

synchronized vs ReentrantLock cheatsheet

Feature	synchronized	ReentrantLock
Timeout on lock	❌	✅ tryLock(n, unit)
Interruptible	❌	✅ lockInterruptibly()
Multiple conditions	❌ 1 implicit	✅ N conditions

Feature	synchronized	ReentrantLock
Fair ordering	❌	✅ new ReentrantLock(true)
Virtual thread safe	❌ pins	✅ no pinning
Auto-release	✅	❌ must unlock() in finally

Execution Flow — AQS (AbstractQueuedSynchronizer)

```
  lock.lock() called by T1, T2, T3:

  T1 ──► CAS(state: 0→1) ──► SUCCESS ──► enters critical section
  T2 ──► CAS(state: 0→1) ──► FAIL ──► enqueue in AQS wait queue ──►
park(T2)
  T3 ──► CAS(state: 0→1) ──► FAIL ──► enqueue in AQS wait queue ──►
park(T3)

  AQS Queue: [HEAD] ↔ [T2-node] ↔ [T3-node] [TAIL]

  T1 calls lock.unlock():
    CAS(state: 1→0)
    unpark(T2)  ← wake head of queue

  T2 wakes ──► CAS(state: 0→1) ──► SUCCESS ──► enters critical section
  T3 still parked in queue
```

Interview One-Liner

> "Use ReentrantLock when you need tryLock with timeout, interruptible locking, multiple Condition objects for precise signalling, or fair ordering. Always unlock() in a finally block — unlike synchronized, JVM won't release it for you. With virtual threads, ReentrantLock avoids carrier-thread pinning."

---

18. SYNCHRONIZED SAME-OBJECT LOCK GOTCHA (70% — TRICKY Q)

The Classic Interview Question

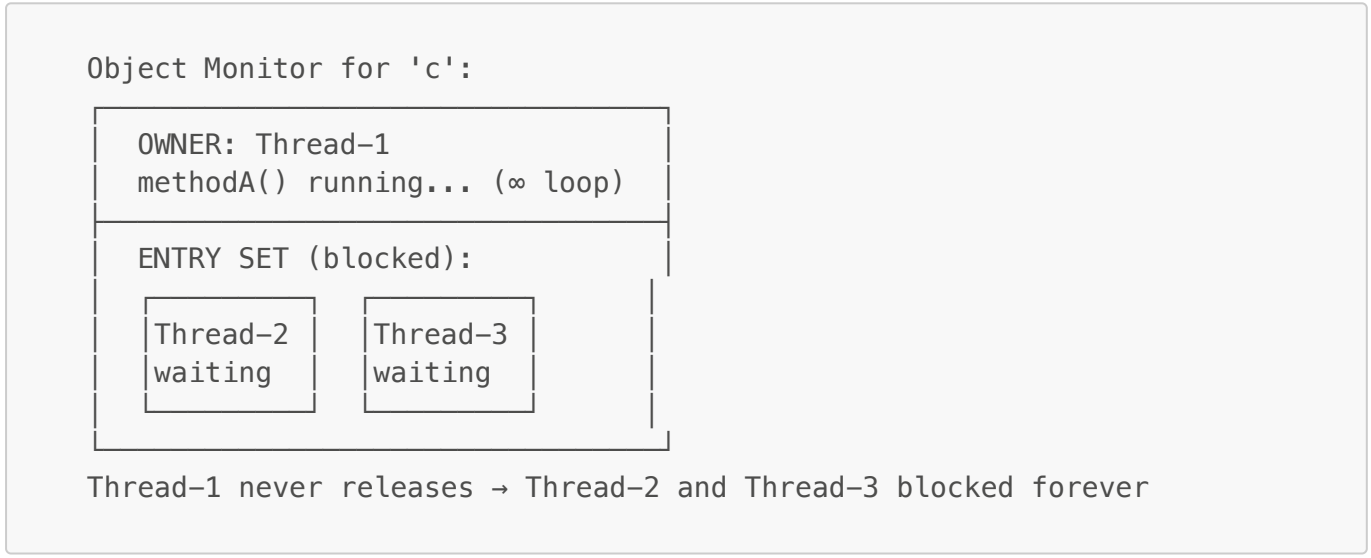
```java
class Counter {
    public synchronized void methodA() {
        System.out.println("A started");
        while (true) { }   // infinite loop
    }

    public synchronized void methodB() {
        System.out.println("B started");
    }
}
```

```
Counter c = new Counter();
new Thread(() -> c.methodA()).start();
new Thread(() -> c.methodB()).start();

// Output:
// A started
// (program hangs — methodB NEVER runs!)
```

Why: Both methods are synchronized on `this`. T1 holds the lock in methodA's infinite loop. T2 blocks forever waiting for the same lock.

Why This Matters

Without understanding this, you design shared objects where one slow method (DB call, external API) starves all other methods including health checks and metrics endpoints. Every `synchronized` method on the same object competes for the same monitor — there is no priority, no timeout, no escape.



Static vs Instance Lock

```
class Demo {
    // Instance method — locks on 'this' (each object has its own lock)
    public synchronized void instanceMethod() { }

    // Static method — locks on Demo.class (ONE lock for entire class)
    public static synchronized void staticMethod() { }
}

Demo d1 = new Demo();
Demo d2 = new Demo();

// d1.instanceMethod() and d2.instanceMethod() → DIFFERENT locks, run in
parallel
// Demo.staticMethod() and Demo.staticMethod() → SAME lock, one blocks the
other
```

```
// d1.instanceMethod() and Demo.staticMethod() → DIFFERENT locks, run in
parallel!
```

The Key Variations

```
// Q: Do these two block each other?
synchronized void syncMethod() { }          // lock = this
void normalMethod() { }                     // NO lock
// Answer: NO — normalMethod has no lock

// Q: Do these two block each other?
synchronized void method1() { }             // lock = this
synchronized void method2() { }             // lock = this
// Answer: YES — same lock object

// Q: Do these block each other across two objects?
d1.method1();   // lock = d1
d2.method1();   // lock = d2
// Answer: NO — different lock objects
```

Interview One-Liner

> "All synchronized instance methods on the same object share ONE lock. If one method holds the lock (even in an infinite loop or slow I/O), ALL other synchronized methods on that object are blocked. Static synchronized methods lock on the Class object — separate from instance locks, but one per class."

---

19. SCOPED VALUES — Java 25 GA (60% — SENIOR SIGNAL)

Why ThreadLocal Breaks with Virtual Threads

```
1 million virtual threads × ThreadLocal copy = heap explosion
InheritableThreadLocal copies value to child → same problem
ThreadLocal is mutable → any code can overwrite it accidentally
Forgetting remove() in thread pools → memory leak + stale data
```

ScopedValue — the fix

```
// Declare (like ThreadLocal but immutable)
static final ScopedValue<User> CURRENT_USER = ScopedValue.newInstance();

// Bind and run — value lives ONLY inside the lambda scope
void handleRequest(User user) throws Exception {
    ScopedValue.where(CURRENT_USER, user)
               .run(() -> processRequest());
```

```
    // CURRENT_USER is unbound here — no cleanup needed
}

void processRequest() {
    User u = CURRENT_USER.get();  // safe anywhere inside the scope
    System.out.println("Serving: " + u.name());
}
```

Multiple bindings

```
ScopedValue.where(CURRENT_USER, user)
           .where(REQUEST_ID, requestId)
           .call(() -> computeResponse());  // call() returns a value
```

With Structured Concurrency — automatic propagation to child tasks

```
ScopedValue.where(CURRENT_USER, user).run(() -> {
    try (var scope = new StructuredTaskScope.ShutdownOnFailure()) {
        scope.fork(() -> {
            CURRENT_USER.get();  // ✅ available in child task
automatically
            return fetchOrders();
        });
        scope.join();
        scope.throwIfFailed();
    }
});
```

Execution Flow

```
  ScopedValue.where(USER, "alice").run(() -> {
  │
  ├──► processRequest()          USER.get() = "alice" ✅
  │      │
  │      └──► StructuredTaskScope.fork(() -> fetchOrders())
  │                │
  │                └──► VT-child   USER.get() = "alice" ✅ (auto-
inherited)
  │
  └── scope exits ──► USER binding removed automatically
  })
  USER.get() here ──► throws NoSuchElementException (unbound)
```

ThreadLocal vs ScopedValue

Aspect	ThreadLocal	ScopedValue
Mutable	✅ yes	❌ immutable
Cleanup required	✅ remove()	❌ auto on scope exit
Virtual thread safe	❌ heap-heavy	✅ efficient
Child thread inherit	Via InheritableThreadLocal	✅ automatic in scope
Java version	All	Java 25 GA

Possible Issues

- `ScopedValue` is immutable per scope — cannot call `.set()` like `ThreadLocal`; need a new `.where()` nesting for a different value.
- Use `orElse(default)` for optional values — `.get()` throws `NoSuchElementException` if not bound.
- Rebinding in a nested scope shadows the outer value (not replaces it) — outer scope still sees original value after inner scope exits.
- Does NOT work with platform threads that predate structured concurrency — the propagation guarantee only holds within `StructuredTaskScope.fork()`.

Interview One-Liner

> "ScopedValue replaces ThreadLocal for virtual threads. It's immutable (no accidental writes), auto-cleaned when scope exits (no leaks), and propagates to child virtual threads within a StructuredTaskScope automatically. ThreadLocal stays relevant for legacy platform-thread code."

TRAP QUESTIONS (things 15 YOE engineers still get wrong)

Trap 1 — Virtual Threads are ALWAYS daemon threads

```
Thread vt = Thread.ofVirtual().start(() -> doWork());
vt.setDaemon(false);  // ❌ throws IllegalThreadStateException (already started)
// Even if set before start: virtual threads ignore it — they are always daemon.

// Consequence: if main() exits, all virtual threads are killed immediately.
// Fix: use try-with-resources on the ExecutorService — it calls awaitTermination:
try (var ex = Executors.newVirtualThreadPerTaskExecutor()) {
    ex.submit(() -> doWork());
}  // blocks here until all tasks finish, THEN closes
```

Trap 2 — synchronized on String or Integer literals

```
// ❌ DANGEROUS — "status" is interned; every class in the JVM sharing
this
//                string literal locks on the SAME object
synchronized ("status") { ... }

// ❌ DANGEROUS — Integer.valueOf(42) is cached (-128 to 127)
//                Two unrelated classes locking on Integer(42) block each
other
synchronized (Integer.valueOf(42)) { ... }

// ✅ Always lock on a private final Object you own
private static final Object LOCK = new Object();
synchronized (LOCK) { ... }
```

Trap 3 — thenApply runs on the COMPLETING thread, not a pool thread

```
CompletableFuture.supplyAsync(() -> fetchFromDB())   // runs on ForkJoin
worker
    .thenApply(data -> heavyTransform(data));         // ❌ runs on SAME
ForkJoin worker
                                                      //    blocks that
worker for heavyTransform

// ✅ Use thenApplyAsync to hand off to a separate thread
    .thenApplyAsync(data -> heavyTransform(data));    // new ForkJoin
worker
    .thenApplyAsync(data -> heavyTransform(data), myExecutor); // custom
pool
```

Trap 4 — parallelStream + blocking I/O = ForkJoinPool deadlock

```
// ForkJoinPool.commonPool() is SHARED across all parallelStreams in the
JVM.
// If tasks block on I/O, all worker threads fill up → deadlock.
list.parallelStream()
    .map(id -> restTemplate.getForObject("/api/" + id, String.class))  //
❌ blocking I/O
    .collect(toList());

// ✅ Use a custom ForkJoinPool to isolate the blocking work
ForkJoinPool custom = new ForkJoinPool(20);
custom.submit(() ->
    list.parallelStream().map(id ->
restTemplate.getForObject(...)).collect(toList())
).get();
```

Trap 5 — volatile array: the reference is volatile, not the elements

```
volatile int[] arr = new int[]{0, 1, 2};

// Another thread writes: arr[0] = 99;
// Your thread reads:     arr[0]        ← MAY see stale value!
// volatile only guarantees visibility of the ARRAY REFERENCE, not its
contents

// ✅ Fix: use AtomicIntegerArray
AtomicIntegerArray atomicArr = new AtomicIntegerArray(3);
atomicArr.set(0, 99);   // atomic + visible
atomicArr.get(0);       // guaranteed fresh
```

Trap 6 — InterruptedException clears the interrupted flag — swallowing it is a bug

```
// ❌ WRONG — flag cleared, caller has no idea the thread was interrupted
try {
    Thread.sleep(1000);
} catch (InterruptedException e) {
    // do nothing — interrupted status is now FALSE
}

// ✅ Option A: re-interrupt so the caller can check
} catch (InterruptedException e) {
    Thread.currentThread().interrupt();  // restore the flag
}

// ✅ Option B: propagate — let the caller handle it
void myMethod() throws InterruptedException {
    Thread.sleep(1000);
}
```

Trap 7 — Static initializer deadlock

```
// Class A and B reference each other's static fields during loading
class A {
    static final int VALUE = B.VALUE + 1;  // triggers B to load
}
class B {
    static final int VALUE = A.VALUE + 1;  // triggers A to load — DEADLOCK
}

// JVM class loading is synchronized per class.
// T1 loads A (holds A's init lock) → needs B → T2 loads B (holds B's init
lock) → needs A → deadlock.
// No stack trace, no exception — threads just hang silently forever.
```

```
// Fix: break the circular dependency; never reference sibling class
statics in static initializers.
```

QUICK-FIRE INTERVIEW ANSWERS

Q: synchronized vs ReentrantLock?

> synchronized is simpler, JVM-managed. ReentrantLock adds: tryLock() with timeout, lockInterruptibly(), fair ordering, multiple Conditions. Use ReentrantLock when you need fine-grained control. With virtual threads, prefer ReentrantLock to avoid pinning.

Q: How to find thread dumps in production?

> `jstack <pid>`, `kill -3 <pid>`, VisualVM, JFR, Spring Boot Actuator `/actuator/threaddump`. Look for "BLOCKED" threads and circular "waiting to lock" chains for deadlocks.

Q: ConcurrentHashMap vs synchronizedMap?

> synchronizedMap wraps HashMap — one lock for entire map. ConcurrentHashMap uses segment/bucket-level locking — concurrent reads never block, writes only lock one bucket. 10-50× more throughput under contention. Also has atomic compound ops: putIfAbsent, compute, merge.

Q: What is a memory barrier?

> Hardware instruction that enforces ordering. Read barrier: flush CPU cache, load from main memory. Write barrier: flush local cache writes to main memory. synchronized and volatile both insert memory barriers.

Q: How does CAS work?

> Compare-And-Swap: hardware instruction. "Set variable to NEW only if current value equals EXPECTED." If another thread changed it first, CAS fails and caller retries (spin loop). AtomicInteger.compareAndSet() uses this. No locks, no blocking — but burns CPU under high contention (→ use LongAdder).

Q: What is false sharing?

> Two variables on same CPU cache line (64 bytes) modified by different threads. Even though variables are independent, each modification invalidates the other thread's cache line. Fix: pad variables to separate cache lines (`@Contended` annotation or manual padding).

Q: Callable vs Runnable?

> Runnable.run() returns void, cannot throw checked exceptions. Callable.call() returns a typed result and can throw checked exceptions. Use Callable when you need the result from a thread.

Q: How do virtual threads differ from reactive programming?

> Both are non-blocking under the hood. Reactive (Reactor/RxJava) requires you to write functional/reactive code with monads — steep learning curve. Virtual threads let you write plain

> blocking Java code — JVM handles the non-blocking aspect. Virtual threads are simpler but don't give backpressure or streaming primitives.

PRODUCTION WAR STORIES (use in interviews)

Story 1 — Thread Pool OOM:

> Used `newCachedThreadPool` for external API calls. Under traffic spike, it created 5,000 threads, JVM OOMed. Fixed by using `ThreadPoolExecutor` with `ArrayBlockingQueue(500)` and `CallerRunsPolicy`. Backpressure propagated to HTTP layer naturally.

Story 2 — Invisible Deadlock:

> Two services called each other via HTTP with same thread pool. Service A held DB lock, called Service B; Service B tried to acquire same DB lock. Deadlock because all threads in pool were blocked. Fixed with lock ordering and timeout on DB operations.

Story 3 — ThreadLocal Memory Leak:

> Request IDs accumulated in thread-local storage of Tomcat thread pool — threads never cleared after reuse. 24-hour uptime → heap grew 2GB. Added `remove()` in filter's finally block. Heap stabilized immediately.

Story 4 — Virtual Thread Pinning:

> Migrated to virtual threads, saw no performance improvement. Diagnosed with `-Djdk.tracePinnedThreads=full` — all blocking I/O inside legacy `synchronized` blocks. Replaced with `ReentrantLock`. Latency dropped 60%.

Last Updated: August 2026 | Targeted: 15 YOE Senior/Staff Java Engineers | Single Print Page